

Namespace BOOSEUnitTesting

Classes

[AppArrayTests](#)

Contains unit tests for the [AppArray](#) class. These tests validate correct parameter parsing, compilation, execution, and array access behavior for both integer and real arrays.

[AppCanvasTests](#)

Contains comprehensive unit tests for [AppCanvas](#).

[AppCommandFactoryTests](#)

Contains unit tests for the [AppCommandFactory](#) class. Verifies that the factory returns the correct command type for each supported command name (case-insensitive and whitespace tolerant) and throws BOOSE.FactoryException for unknown commands.

[AppElseTests](#)

[AppEndForTests](#)

Unit tests for [AppEndFor](#). These tests validate parameter rules, runtime stack behaviour, and loop-finish vs loop-continue decisions without asserting on StoredProgram's runner-controlled PC behaviour.

[AppEndIfTests](#)

Unit tests for [AppEndIf](#). Validates parameter rules and runtime stack behaviour when ending an if-block.

[AppEndWhileTests](#)

Unit tests for [AppEndWhile](#). These tests validate runtime-stack behaviour and error handling. PC-jump behaviour is intentionally not asserted here because it is runner/integration behaviour and can vary depending on StoredProgram implementation.

[AppForTests](#)

Unit tests for [AppFor](#). Validates syntax checking, auto-variable creation, compile-time type validation, execution behaviour across iterations, and step handling.

[AppIfTests](#)

Unit tests for [AppIf](#). Focuses on condition evaluation and branch decision logic without relying on StoredProgram's execution loop.

[AppIntTests](#)

Contains unit tests for the [AppInt](#) command. Tests cover parameter validation, compile-time validation (variable existence), and execution-time evaluation/updating of an integer variable.

[AppPeekTests](#)

Contains unit tests for the [AppPeek](#) command. Tests cover parameter validation/parsing, compile-time validation (variable/array existence), and execution-time behavior for reading values from [AppArray](#) instances.

[AppPokeTests](#)

Contains unit tests for the [AppPoke](#) command. Tests cover parameter validation/parsing, compile-time validation (array existence), and execution-time behavior for writing values into [AppArray](#) elements.

[AppRealTests](#)

Contains unit tests for the [AppReal](#) command. These tests focus on parameter validation and compile-time validation, and account for BOOSE runtime typing rules which require a true "real" variable to successfully call [UpdateVariable\(string, double\)](#).[↗](#)

[AppWhileTests](#)

Unit tests for [AppWhile](#). Validates condition evaluation, operator validation, and loop exit behaviour without relying on program runner.

[CircleCommandTests](#)

Contains unit tests for the [CircleCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal radius values.

[CommandParserTests](#)

Unit tests for [CommandParser](#). Covers parsing of single-line commands (including special-cased "end if/while/for"), variable declarations (int/real), assignments, and loop/if structure validation.

[DrawToCommandTests](#)

Contains unit tests for the [DrawToCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

[MoveToCommandTests](#)

Contains unit tests for the [MoveToCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

[PenCommandTests](#)

Contains unit tests for the [PenCommand](#) class. Tests cover parameter validation, compile-time validation for literal RGB values, and execution-time behavior including runtime range checking.

[RectangleCommandTests](#)

Contains unit tests for the [RectangleCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

[TestBase](#)

Provides common setup for BOOSE command unit tests, including a BOOSE.StoredProgram created with a headless BOOSE.ICanvas.

[WriteCommandTests](#)

Contains unit tests for the [WriteCommand](#) class. Tests cover parameter validation and execution behavior for both evaluated text and literal (fallback) text.

Class AppArrayTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppArray](#) class. These tests validate correct parameter parsing, compilation, execution, and array access behavior for both integer and real arrays.

```
[TestClass]  
public class AppArrayTests
```

Inheritance

[object](#)  ← AppArrayTests

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotThree_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when the number of parameters is not exactly three.

```
[TestMethod]  
public void CheckParameters_WhenNotThree_ThrowsParserException()
```

CheckParameters_WhenThree_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw an exception when exactly three parameters are supplied.

```
[TestMethod]  
public void CheckParameters_WhenThree_DoesNotThrow()
```

Compile_WhenSizeLessThanOne_ThrowsParserException()

Ensures that [Compile\(\)](#) throws a BOOSE.ParserException when the array size is less than one.

```
[TestMethod]
public void Compile_WhenSizeLessThanOne_ThrowsParserException()
```

Compile_WhenTypeInvalid_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when an unsupported array type is used.

```
[TestMethod]
public void Compile_WhenTypeInvalid_ThrowsParserException()
```

Compile_WhenValid_AddsVariableToStoredProgram()

Confirms that a valid array declaration is registered in the BOOSE.StoredProgram during compilation.

```
[TestMethod]
public void Compile_WhenValid_AddsVariableToStoredProgram()
```

Execute_WhenInt_CreatesIntArrayOfCorrectSize()

Verifies that executing an integer array declaration creates an integer array of the expected size.

```
[TestMethod]
public void Execute_WhenInt_CreatesIntArrayOfCorrectSize()
```

Execute_WhenReal_CreatesRealArrayOfCorrectSize()

Verifies that executing a real array declaration creates a real array of the expected size.

```
[TestMethod]
public void Execute_WhenReal_CreatesRealArrayOfCorrectSize()
```

SetInt_ThenGetInt_ReturnsSameValue()

Ensures that setting and retrieving an integer value from an array returns the correct result.

```
[TestMethod]
public void SetInt_ThenGetInt_ReturnsSameValue()
```

SetReal_WhenIndexOutOfBounds_ThrowsCommandException()

Ensures that accessing an array index outside its bounds throws a BOOSE.CommandException.

```
[TestMethod]
public void SetReal_WhenIndexOutOfBounds_ThrowsCommandException()
```

Set_ParsesTypeNameSize_AndSetsProperties()

Ensures that the [Set\(StoredProgram, string\)](#) method correctly parses the array type, variable name, and size.

```
[TestMethod]
public void Set_ParsesTypeNameSize_AndSetsProperties()
```

Set_TypeIsLowercased()

Verifies that the array type parsed by [Set\(StoredProgram, string\)](#) is normalized to lowercase.

```
[TestMethod]
public void Set_TypeIsLowercased()
```

Set_WhenParameterCountWrong_ThrowsParserException()

Confirms that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is incorrect.

```
[TestMethod]
public void Set_WhenParameterCountWrong_ThrowsParserException()
```

Set_WhenSizeNotInt_ThrowsFormatException()

Ensures that [Set\(StoredProgram, string\)](#) throws a [FormatException](#) when the array size is not a valid integer.

```
[TestMethod]
public void Set_WhenSizeNotInt_ThrowsFormatException()
```

Setup()

Initializes a fresh BOOSE.StoredProgram before each test to ensure test isolation and avoid shared state.

```
[TestInitialize]
public void Setup()
```

Class AppCanvasTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains comprehensive unit tests for [AppCanvas](#).

```
[TestClass]
public class AppCanvasTests
```

Inheritance

[object](#)  ← AppCanvasTests

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Remarks

Some methods and property setters in [AppCanvas](#) catch `BOOSE.CanvasException` and display a modal `MessageBox` (instead of rethrowing). These tests include invalid-input cases that will display `MessageBoxes`; close them manually during the run as desired.

Methods

Circle_DrawsOnBitmap()

Verifies that [Circle\(int, bool\)](#) draws non-white pixels on the bitmap.

```
[TestMethod]
public void Circle_DrawsOnBitmap()
```

Clear_ResetsBitmapToWhite()

Verifies that [Clear\(\)](#) makes the bitmap white everywhere (sampled).

```
[TestMethod]
```

```
public void Clear_ResetsBitmapToWhite()
```

Constructor_SetsDimensions_AndCreatesBitmap()

Verifies the constructor sets width/height and produces a bitmap with matching dimensions.

```
[TestMethod]  
public void Constructor_SetsDimensions_AndCreatesBitmap()
```

DrawTo_WhenNegativeX_DoesNotUpdatePosition()

Verifies that [DrawTo\(int, int\)](#) rejects negative X (shows MessageBox) and does not update position.

```
[TestMethod]  
public void DrawTo_WhenNegativeX_DoesNotUpdatePosition()
```

DrawTo_WhenNegativeY_DoesNotUpdatePosition()

Verifies that [DrawTo\(int, int\)](#) rejects negative Y (shows MessageBox) and does not update position.

```
[TestMethod]  
public void DrawTo_WhenNegativeY_DoesNotUpdatePosition()
```

DrawTo_WhenValid_DrawsAndUpdatesPosition()

Verifies that [DrawTo\(int, int\)](#) draws a line and updates pen position.

```
[TestMethod]  
public void DrawTo_WhenValid_DrawsAndUpdatesPosition()
```

DrawTo_WhenXGreaterThanWidth_DoesNotUpdatePosition()

Verifies that [DrawTo\(int, int\)](#) rejects $X > \text{width}$ (shows MessageBox) and does not update position.

```
[TestMethod]
public void DrawTo_WhenXGreaterThanWidth_DoesNotUpdatePosition()
```

DrawTo_WhenYGreaterThanHeight_DoesNotUpdatePosition()

Verifies that [DrawTo\(int, int\)](#) rejects $Y > \text{height}$ (shows MessageBox) and does not update position.

```
[TestMethod]
public void DrawTo_WhenYGreaterThanHeight_DoesNotUpdatePosition()
```

Draw_Method_ThrowsNotImplementedException()

Verifies that [draw\(Graphics\)](#) throws [NotImplementedException](#).

```
[TestMethod]
public void Draw_Method_ThrowsNotImplementedException()
```

MoveTo_WhenNegativeX_DoesNotUpdatePosition()

Verifies that [MoveTo\(int, int\)](#) rejects negative X (shows MessageBox) and does not update position.

```
[TestMethod]
public void MoveTo_WhenNegativeX_DoesNotUpdatePosition()
```

MoveTo_WhenNegativeY_DoesNotUpdatePosition()

Verifies that [MoveTo\(int, int\)](#) rejects negative Y (shows MessageBox) and does not update position.

```
[TestMethod]
public void MoveTo_WhenNegativeY_DoesNotUpdatePosition()
```

MoveTo_WhenValid_UpdatesPosition()

Verifies that [MoveTo\(int, int\)](#) updates pen position for valid coordinates.

```
[TestMethod]
public void MoveTo_WhenValid_UpdatesPosition()
```

MoveTo_WhenXGreaterThanWidth_DoesNotUpdatePosition()

Verifies that [MoveTo\(int, int\)](#) rejects $X > \text{width}$ (shows MessageBox) and does not update position.

```
[TestMethod]
public void MoveTo_WhenXGreaterThanWidth_DoesNotUpdatePosition()
```

MoveTo_WhenYGreaterThanHeight_DoesNotUpdatePosition()

Verifies that [MoveTo\(int, int\)](#) rejects $Y > \text{height}$ (shows MessageBox) and does not update position.

```
[TestMethod]
public void MoveTo_WhenYGreaterThanHeight_DoesNotUpdatePosition()
```

Rect_DrawsOnBitmap()

Verifies that [Rect\(int, int, bool\)](#) draws non-white pixels on the bitmap.

```
[TestMethod]
public void Rect_DrawsOnBitmap()
```

SetColour_WhenComponentOver255_ThrowsCanvasException()

Verifies that [SetColour\(int, int, int\)](#) throws when any component is greater than 255.

```
[TestMethod]
public void SetColour_WhenComponentOver255_ThrowsCanvasException()
```

SetColour_WhenValid_UpdatesPenColour_AndAffectsDrawing()

Verifies that [SetColour\(int, int, int\)](#) sets [PenColour](#) and that subsequent drawing uses that colour.

```
[TestMethod]  
public void SetColour_WhenValid_UpdatesPenColour_AndAffectsDrawing()
```

Set_DoesNotThrow_AndBitmapStillDrawable()

Verifies that [Set\(int, int\)](#) does not throw and the bitmap remains usable.

```
[TestMethod]  
public void Set_DoesNotThrow_AndBitmapStillDrawable()
```

Tri_DrawsOnBitmap()

Verifies that [Tri\(int, int\)](#) draws non-white pixels on the bitmap.

```
[TestMethod]  
public void Tri_DrawsOnBitmap()
```

WriteText_DrawsOnBitmap()

Verifies that [WriteText\(string\)](#) changes the bitmap (non-white samples appear).

```
[TestMethod]  
public void WriteText_DrawsOnBitmap()
```

XposYpos_WhenInBounds_SetsValues()

Verifies that setting [Xpos](#) and [Ypos](#) within bounds works.

```
[TestMethod]  
public void XposYpos_WhenInBounds_SetsValues()
```

Xpos_WhenGreaterThanWidth_DoesNotChangeValue()

Verifies that setting [Xpos](#) greater than width is rejected (shows MessageBox) and does not change the stored position.

```
[TestMethod]  
public void Xpos_WhenGreaterThanWidth_DoesNotChangeValue()
```

Xpos_WhenNegative_DoesNotChangeValue()

Verifies that setting [Xpos](#) to a negative value is rejected (shows MessageBox) and does not change the stored position.

```
[TestMethod]  
public void Xpos_WhenNegative_DoesNotChangeValue()
```

Ypos_WhenGreaterThanHeight_DoesNotChangeValue()

Verifies that setting [Ypos](#) greater than height is rejected (shows MessageBox) and does not change the stored position.

```
[TestMethod]  
public void Ypos_WhenGreaterThanHeight_DoesNotChangeValue()
```

Ypos_WhenNegative_DoesNotChangeValue()

Verifies that setting [Ypos](#) to a negative value is rejected (shows MessageBox) and does not change the stored position.

```
[TestMethod]  
public void Ypos_WhenNegative_DoesNotChangeValue()
```

Class AppCommandFactoryTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppCommandFactory](#) class. Verifies that the factory returns the correct command type for each supported command name (case-insensitive and whitespace tolerant) and throws BOOSE.FactoryException for unknown commands.

```
[TestClass]
public class AppCommandFactoryTests
```

Inheritance

[object](#)  ← AppCommandFactoryTests

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

MakeCommand_WhenArray_ReturnsAppArray()

Verifies that the factory returns an [AppArray](#) for "array".

```
[TestMethod]
public void MakeCommand_WhenArray_ReturnsAppArray()
```

MakeCommand_WhenCircle_ReturnsCircleCommand()

Verifies that the factory returns a [CircleCommand](#) for "circle".

```
[TestMethod]
public void MakeCommand_WhenCircle_ReturnsCircleCommand()
```

MakeCommand_WhenDrawTo_ReturnsDrawToCommand()

Verifies that the factory returns a [DrawToCommand](#) for "drawto".

```
[TestMethod]
public void MakeCommand_WhenDrawTo_ReturnsDrawToCommand()
```

MakeCommand_WhenElse_ReturnsAppElse()

Verifies that the factory returns an [AppElse](#) for "else".

```
[TestMethod]
public void MakeCommand_WhenElse_ReturnsAppElse()
```

MakeCommand_WhenEndFor_ReturnsAppEndFor()

Verifies that the factory returns an [AppEndFor](#) for "endfor".

```
[TestMethod]
public void MakeCommand_WhenEndFor_ReturnsAppEndFor()
```

MakeCommand_WhenEndIf_ReturnsAppEndIf()

Verifies that the factory returns an [AppEndIf](#) for "endif".

```
[TestMethod]
public void MakeCommand_WhenEndIf_ReturnsAppEndIf()
```

MakeCommand_WhenEndWhile_ReturnsAppEndWhile()

Verifies that the factory returns an [AppEndWhile](#) for "endwhile".

```
[TestMethod]
public void MakeCommand_WhenEndWhile_ReturnsAppEndWhile()
```

MakeCommand_WhenFor_ReturnsAppFor()

Verifies that the factory returns an [AppFor](#) for "for".

```
[TestMethod]
public void MakeCommand_WhenFor_ReturnsAppFor()
```

MakeCommand_WhenIf_ReturnsAppIf()

Verifies that the factory returns an [AppIf](#) for "if".

```
[TestMethod]
public void MakeCommand_WhenIf_ReturnsAppIf()
```

MakeCommand_WhenInt_ReturnsAppInt()

Verifies that the factory returns an [AppInt](#) for "int".

```
[TestMethod]
public void MakeCommand_WhenInt_ReturnsAppInt()
```

MakeCommand_WhenMoveTo_ReturnsMoveToCommand()

Verifies that the factory returns a [MoveToCommand](#) for "moveto".

```
[TestMethod]
public void MakeCommand_WhenMoveTo_ReturnsMoveToCommand()
```

MakeCommand_WhenPeek_ReturnsAppPeek()

Verifies that the factory returns an [AppPeek](#) for "peek".

```
[TestMethod]
public void MakeCommand_WhenPeek_ReturnsAppPeek()
```

MakeCommand_WhenPen_ReturnsPenCommand()

Verifies that the factory returns a [PenCommand](#) for "pen".

```
[TestMethod]
public void MakeCommand_WhenPen_ReturnsPenCommand()
```

MakeCommand_WhenPoke_ReturnsAppPoke()

Verifies that the factory returns an [AppPoke](#) for "poke".

```
[TestMethod]
public void MakeCommand_WhenPoke_ReturnsAppPoke()
```

MakeCommand_WhenReal_ReturnsAppReal()

Verifies that the factory returns an [AppReal](#) for "real".

```
[TestMethod]
public void MakeCommand_WhenReal_ReturnsAppReal()
```

MakeCommand_WhenRect_ReturnsRectangleCommand()

Verifies that the factory returns a [RectangleCommand](#) for "rect".

```
[TestMethod]
public void MakeCommand_WhenRect_ReturnsRectangleCommand()
```

MakeCommand_WhenUnknown_ThrowsFactoryException()

Verifies that an unsupported command name throws BOOSE.FactoryException.

```
[TestMethod]
public void MakeCommand_WhenUnknown_ThrowsFactoryException()
```

MakeCommand_WhenWhile_ReturnsAppWhile()

Verifies that the factory returns an [AppWhile](#) for "while".

```
[TestMethod]  
public void MakeCommand_WhenWhile_ReturnsAppWhile()
```

MakeCommand_WhenWrite_ReturnsWriteCommand()

Verifies that the factory returns a [WriteCommand](#) for "write".

```
[TestMethod]  
public void MakeCommand_WhenWrite_ReturnsWriteCommand()
```

Class AppElseTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

```
[TestClass]  
public class AppElseTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#)  ← AppElseTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenParameterProvided_ThrowsParserException()

```
[TestMethod]  
public void CheckParameters_WhenParameterProvided_ThrowsParserException()
```

Execute_WhenIfConditionFalse_DoesNotSkipElseBody()

```
[TestMethod]  
public void Execute_WhenIfConditionFalse_DoesNotSkipElseBody()
```

Execute_WhenIfConditionTrue_SkipsElseBodyBySettingProgram Counter()

```
[TestMethod]
```

```
public void Execute_WhenIfConditionTrue_SkipsElseBodyBySettingProgramCounter()
```

Execute_WhenNoMatchingIfOnStack_ThrowsCommandException()

```
[TestMethod]  
public void Execute_WhenNoMatchingIfOnStack_ThrowsCommandException()
```

Set_WhenNoParameters_DoesNotThrow()

```
[TestMethod]  
public void Set_WhenNoParameters_DoesNotThrow()
```

Setup()

```
[TestInitialize]  
public void Setup()
```

Class AppEndForTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppEndFor](#). These tests validate parameter rules, runtime stack behaviour, and loop-finish vs loop-continue decisions without asserting on StoredProgram's runner-controlled PC behaviour.

```
[TestClass]
[DoNotParallelize]
public class AppEndForTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppEndForTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenParameterProvided_ThrowsParserException()

Verifies that providing any parameter causes BOOSE.ParserException.

```
[TestMethod]
public void CheckParameters_WhenParameterProvided_ThrowsParserException()
```

Execute_WhenLoopFinished_NegativeStep_PopsFor()

Verifies that when the loop is finished (negative step: currentValue < EndValue), end-for pops the loop from the runtime stack.

```
[TestMethod]
```

```
public void Execute_WhenLoopFinished_NegativeStep_PopsFor()
```

Execute_WhenLoopFinished_PositiveStep_PopsFor()

Verifies that when the loop is finished (positive step: currentValue > EndValue), end-for pops the loop from the runtime stack.

```
[TestMethod]  
public void Execute_WhenLoopFinished_PositiveStep_PopsFor()
```

Execute_WhenLoopNotFinished_DoesNotPopFor()

Verifies that when the loop is not finished, end-for does not pop the loop (it will request a jump back to the for line via PC in the real runner).

```
[TestMethod]  
public void Execute_WhenLoopNotFinished_DoesNotPopFor()
```

Execute_WhenNoMatchingForOnStack_ThrowsParserException()

Verifies that executing end-for with an empty runtime stack throws. Note: the current implementation throws BOOSE.ParserException (not CommandException).

```
[TestMethod]  
public void Execute_WhenNoMatchingForOnStack_ThrowsParserException()
```

Set_WhenNoParameters_DoesNotThrow()

Verifies that setting with no parameters succeeds.

```
[TestMethod]  
public void Set_WhenNoParameters_DoesNotThrow()
```

Setup()

```
[TestInitialize]  
public void Setup()
```

Class AppEndIfTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppEndIf](#). Validates parameter rules and runtime stack behaviour when ending an if-block.

```
[TestClass]
public class AppEndIfTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppEndIfTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenParameterProvided_ThrowsParserException()

Verifies that providing any parameter causes BOOSE.ParserException.

```
[TestMethod]
public void CheckParameters_WhenParameterProvided_ThrowsParserException()
```

Execute_WhenIfOnStack_PopsIf()

Verifies that executing end-if pops the most recent IF from the runtime stack.

```
[TestMethod]
public void Execute_WhenIfOnStack_PopsIf()
```

Execute_WhenNoMatchingIfOnStack_ThrowsCommandException()

Verifies that executing end-if with an empty runtime stack throws BOOSE.CommandException.

```
[TestMethod]
public void Execute_WhenNoMatchingIfOnStack_ThrowsCommandException()
```

Set_WhenNoParameters_DoesNotThrow()

Verifies that setting with no parameters succeeds.

```
[TestMethod]
public void Set_WhenNoParameters_DoesNotThrow()
```

Setup()

Clears the IF runtime stack before each test to prevent static-state leakage.

```
[TestInitialize]
public void Setup()
```

Class AppEndWhileTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppEndWhile](#). These tests validate runtime-stack behaviour and error handling. PC-jump behaviour is intentionally not asserted here because it is runner/integration behaviour and can vary depending on StoredProgram implementation.

```
[TestClass]
[DoNotParallelize]
public class AppEndWhileTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppEndWhileTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

Execute_WhenNoMatchingWhileOnStack_ThrowsCommandException()

Verifies that executing with an empty runtime stack throws BOOSE.CommandException.

```
[TestMethod]
public void Execute_WhenNoMatchingWhileOnStack_ThrowsCommandException()
```

Execute_WhenWhileConditionFalse_PopsWhile()

Verifies that end-while pops a while command when one exists (condition false).

```
[TestMethod]
```

```
public void Execute_WhenWhileConditionFalse_PopsWhile()
```

Execute_WhenWhileConditionTrue_PopsWhile()

Verifies that end-while pops a while command when one exists (condition true).

```
[TestMethod]  
public void Execute_WhenWhileConditionTrue_PopsWhile()
```

Setup()

```
[TestInitialize]  
public void Setup()
```

Class AppForTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppFor](#). Validates syntax checking, auto-variable creation, compile-time type validation, execution behaviour across iterations, and step handling.

```
[TestClass]
public class AppForTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppForTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenInvalidSyntax_ThrowsParserException()

Ensures invalid FOR parameter counts/syntax throw BOOSE.ParserException.

```
[TestMethod]
public void CheckParameters_WhenInvalidSyntax_ThrowsParserException()
```

Compile_WhenVariableIsNotInt_ThrowsParserException()

Verifies compile fails if loop variable is not an integer.

```
[TestMethod]
public void Compile_WhenVariableIsNotInt_ThrowsParserException()
```

Execute_FirstIteration_SetsStartValue()

Verifies first execution initializes loop variable to the start value.

```
[TestMethod]
public void Execute_FirstIteration_SetsStartValue()
```

Execute_SubsequentIteration_IncrementsByStep()

Verifies subsequent executions increment the loop variable by the step value.

```
[TestMethod]
public void Execute_SubsequentIteration_IncrementsByStep()
```

Execute_WhenLoopExits_NegativeStep_AdvancesPastEndValue()

Ensures the loop completes when it passes the end bound (negative step).

```
[TestMethod]
public void Execute_WhenLoopExits_NegativeStep_AdvancesPastEndValue()
```

Execute_WhenLoopExits_PositiveStep_AdvancesPastEndValue()

Ensures the loop completes when it passes the end bound (positive step).

```
[TestMethod]
public void Execute_WhenLoopExits_PositiveStep_AdvancesPastEndValue()
```

Remarks

In unit tests we do not assert on `BOOSE.StoredProgram.PC` because that is controlled by the program runner. Instead we assert that the loop advances past the end value and marks itself as started.

Execute_WhenStepsZero_ThrowsCommandException()

Ensures step of zero throws BOOSE.CommandException.

```
[TestMethod]  
public void Execute_WhenStepIsZero_ThrowsCommandException()
```

Set_WhenVariableDoesNotExist_AutoCreatesIntVariable()

Ensures a FOR loop auto-creates its control variable if missing.

```
[TestMethod]  
public void Set_WhenVariableDoesNotExist_AutoCreatesIntVariable()
```

Class AppIfTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppIf](#). Focuses on condition evaluation and branch decision logic without relying on StoredProgram's execution loop.

```
[TestClass]
public class AppIfTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppIfTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

Compile_WhenOperatorInvalid_ThrowsParserException()

Ensures invalid operators throw at compile time.

```
[TestMethod]
public void Compile_WhenOperatorInvalid_ThrowsParserException()
```

EvaluateCondition_WhenFalse_ReturnsFalse()

Verifies EvaluateCondition returns false for valid comparisons.

```
[TestMethod]
public void EvaluateCondition_WhenFalse_ReturnsFalse()
```

EvaluateCondition_WhenTrue_ReturnsTrue()

Verifies EvaluateCondition returns true for valid comparisons.

```
[TestMethod]  
public void EvaluateCondition_WhenTrue_ReturnsTrue()
```

Execute_WhenConditionFalse_SetsConditionFalse()

Ensures Execute sets BOOSE.ConditionalCommand.Condition to false when the condition is false.

```
[TestMethod]  
public void Execute_WhenConditionFalse_SetsConditionFalse()
```

Execute_WhenConditionTrue_SetsConditionTrue()

Ensures Execute sets BOOSE.ConditionalCommand.Condition to true when the condition is true.

```
[TestMethod]  
public void Execute_WhenConditionTrue_SetsConditionTrue()
```

Class AppIntTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppInt](#) command. Tests cover parameter validation, compile-time validation (variable existence), and execution-time evaluation/updating of an integer variable.

```
[TestClass]
public class AppIntTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#)  ← AppIntTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotOne_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when the parameter count is not exactly one.

```
[TestMethod]
public void CheckParameters_WhenNotOne_ThrowsParserException()
```

CheckParameters_WhenOne_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly one parameter is supplied.

```
[TestMethod]
public void CheckParameters_WhenOne_DoesNotThrow()
```

Compile_WhenVariableDoesNotExist_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the variable does not exist in the BOOSE.StoredProgram.

```
[TestMethod]
public void Compile_WhenVariableDoesNotExist_ThrowsParserException()
```

Compile_WhenVariableExists_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when the variable exists.

```
[TestMethod]
public void Compile_WhenVariableExists_DoesNotThrow()
```

Compile_WhenVariableNameNotSet_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the variable name has not been set.

```
[TestMethod]
public void Compile_WhenVariableNameNotSet_ThrowsParserException()
```

Execute_WhenResultNotInt_ThrowsFormatException()

Verifies that [Execute\(\)](#) throws when the evaluated expression cannot be parsed as an integer.

```
[TestMethod]
public void Execute_WhenResultNotInt_ThrowsFormatException()
```

Execute_WhenValid_UpdatesVariableValue()

Verifies that [Execute\(\)](#) evaluates the expression at runtime and updates the target integer variable in the program.

```
[TestMethod]
public void Execute_WhenValid_UpdatesVariableValue()
```

Remarks

This test uses subtraction because in your BOOSE build it evaluates numerically, while "+" may be treated as concatenation in some cases.

Set_StoresTrimmedExpression_DoesNotThrow()

Verifies that [Set\(StoredProgram, string\)](#) stores a trimmed expression without throwing.

```
[TestMethod]
public void Set_StoresTrimmedExpression_DoesNotThrow()
```

Class AppPeekTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppPeek](#) command. Tests cover parameter validation/parsing, compile-time validation (variable/array existence), and execution-time behavior for reading values from [AppArray](#) instances.

```
[TestClass]
public class AppPeekTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppPeekTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Remarks

BOOSE enforces strict runtime typing for [UpdateVariable\(string, int\)](#)  and [UpdateVariable\(string, double\)](#) . In this project, registering a variable name via `BOOSE.StoredProgram.AddVariable(BOOSE.Evaluation)` does not make it a typed BOOSE int/real variable, so execution tests that attempt to update a scalar variable will typically throw `BOOSE.CommandException` (type mismatch).

The execution tests below therefore focus on:

- validation paths (missing array/variable, not-an-array, out-of-bounds)
- successful retrieval from the array (verified by the array still holding the expected value)
- and the fact that a scalar update attempt occurs (observed via the expected BOOSE type-mismatch exception).

Methods

CheckParameters_WhenFourOrMore_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when at least four parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenFourOrMore_DoesNotThrow()
```

CheckParameters_WhenLessThanFour_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when fewer than four parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenLessThanFour_ThrowsParserException()
```

Compile_WhenArrayMissing_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the array does not exist.

```
[TestMethod]
public void Compile_WhenArrayMissing_ThrowsParserException()
```

Compile_WhenTargetAndArrayExist_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when the target variable and array both exist.

```
[TestMethod]
public void Compile_WhenTargetAndArrayExist_DoesNotThrow()
```

Compile_WhenTargetVariableMissing_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the target variable does not exist.

```
[TestMethod]
public void Compile_WhenTargetVariableMissing_ThrowsParserException()
```

Execute_WhenIndexOutOfBounds_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the requested index is out of bounds for the target array.

```
[TestMethod]
public void Execute_WhenIndexOutOfBounds_ThrowsCommandException()
```

Execute_WhenIntArray_AttemptsUpdate_ThrowsTypeMismatchCommandException()

Verifies that [Execute\(\)](#) attempts to peek from an int array and then update the target variable. In this BOOSE build, updating a non-typed dummy variable will throw a type-mismatch BOOSE.CommandException.

```
[TestMethod]
public void Execute_WhenIntArray_AttemptsUpdate_ThrowsTypeMismatchCommandException()
```

Execute_WhenRealArray_AttemptsUpdate_ThrowsTypeMismatchCommandException()

Verifies that [Execute\(\)](#) attempts to peek from a real array and then update the target variable. In this BOOSE build, updating a non-typed dummy variable will throw a type-mismatch BOOSE.CommandException.

```
[TestMethod]
public void Execute_WhenRealArray_AttemptsUpdate_ThrowsTypeMismatchCommandException()
```

Execute_WhenReferencedVariableNotArray_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the referenced "array" name exists but is not an [AppArray](#).

```
[TestMethod]
```

```
public void Execute_WhenReferencedVariableNotArray_ThrowsCommandException()
```

Set_WhenTooFewTokens_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when fewer than four tokens are supplied.

```
[TestMethod]
```

```
public void Set_WhenTooFewTokens_ThrowsParserException()
```

Remarks

In the current implementation, [CheckParameters\(string\[\]\)](#) is called before indexing into the token array, so the expected exception is BOOSE.ParserException.

Class AppPokeTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppPoke](#) command. Tests cover parameter validation/parsing, compile-time validation (array existence), and execution-time behavior for writing values into [AppArray](#) elements.

```
[TestClass]  
public class AppPokeTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppPokeTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenFourOrMore_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when at least four parameters are supplied.

```
[TestMethod]  
public void CheckParameters_WhenFourOrMore_DoesNotThrow()
```

CheckParameters_WhenLessThanFour_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a [BOOSE.ParserException](#) when fewer than four parameters are supplied.

```
[TestMethod]  
public void CheckParameters_WhenLessThanFour_ThrowsParserException()
```

Compile_WhenArrayExists_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when the referenced array exists.

```
[TestMethod]
public void Compile_WhenArrayExists_DoesNotThrow()
```

Compile_WhenArrayMissing_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the referenced array does not exist.

```
[TestMethod]
public void Compile_WhenArrayMissing_ThrowsParserException()
```

Execute_WhenIndexOutOfBounds_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the evaluated index is out of bounds for the target array.

```
[TestMethod]
public void Execute_WhenIndexOutOfBounds_ThrowsCommandException()
```

Execute_WhenIntArray_PokesValueIntoArray()

Verifies that [Execute\(\)](#) writes an integer value into an int array element.

```
[TestMethod]
public void Execute_WhenIntArray_PokesValueIntoArray()
```

Execute_WhenRealArray_PokesValueIntoArray()

Verifies that [Execute\(\)](#) writes a real value into a real array element.

```
[TestMethod]
```

```
public void Execute_WhenRealArray_PokesValueIntoArray()
```

Execute_WhenReferencedVariableNotArray_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the referenced variable exists but is not an [AppArray](#).

```
[TestMethod]
public void Execute_WhenReferencedVariableNotArray_ThrowsCommandException()
```

Execute_WhenValueIsExpression_EvaluatesAndPokes()

Verifies that [Execute\(\)](#) evaluates expressions in the value parameter using [EvaluateExpressionWithString\(string\)](#).

```
[TestMethod]
public void Execute_WhenValueIsExpression_EvaluatesAndPokes()
```

Remarks

Uses subtraction because in your BOOSE build it evaluates numerically, while "+" may be treated as concatenation in some cases.

Set_WhenTooFewTokens_ThrowsIndexOutOfRangeException()

Verifies that [Set\(StoredProgram, string\)](#) throws an [IndexOutOfRangeException](#) when too few tokens are supplied.

```
[TestMethod]
public void Set_WhenTooFewTokens_ThrowsIndexOutOfRangeException()
```

Remarks

This matches the current implementation: `paramParts[3]` is accessed before [CheckParameters\(string\[\]\)](#) is called.

Class AppRealTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [AppReal](#) command. These tests focus on parameter validation and compile-time validation, and account for BOOSE runtime typing rules which require a true "real" variable to successfully call [UpdateVariable\(string, double\)](#).

```
[TestClass]
public class AppRealTests : TestBase
```

Inheritance

[object](#) ← [TestBase](#) ← AppRealTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#)

Remarks

In this BOOSE build, registering a variable name via `BOOSE.StoredProgram.AddVariable(BOOSE.Evaluation)` does not make it a typed "real" variable. As a result, [Execute\(\)](#) will throw a `BOOSE.CommandException` ("Type mismatch, expected a real value") when it attempts to update the variable. The execution tests below assert that expected behavior.

Methods

CheckParameters_WhenNotOne_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a `BOOSE.ParserException` when the parameter count is not exactly one.

```
[TestMethod]
public void CheckParameters_WhenNotOne_ThrowsParserException()
```

CheckParameters_WhenOne_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly one parameter is supplied.

```
[TestMethod]
public void CheckParameters_WhenOne_DoesNotThrow()
```

Compile_WhenVariableDoesNotExist_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the target variable does not exist in the BOOSE.StoredProgram.

```
[TestMethod]
public void Compile_WhenVariableDoesNotExist_ThrowsParserException()
```

Compile_WhenVariableExists_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when the target variable exists.

```
[TestMethod]
public void Compile_WhenVariableExists_DoesNotThrow()
```

Compile_WhenVariableNameNotSet_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when no variable name has been provided via [SetVariableName\(string\)](#).

```
[TestMethod]
public void Compile_WhenVariableNameNotSet_ThrowsParserException()
```

Execute_WhenExpressionNotNumeric_ThrowsCommandExceptionDueToTypeMismatch()

Verifies that [Execute\(\)](#) still throws a BOOSE.CommandException for a non-numeric expression when the target is not a typed BOOSE real variable.

```
[TestMethod]
public void Execute_WhenExpressionNotNumeric_ThrowsCommandExceptionDueToTypeMismatch()
```

Remarks

In a fully typed BOOSE environment, this scenario might throw [FormatException](#) from [Parse\(string\)](#). In this project, BOOSE throws earlier due to type mismatch.

Execute_WhenVariableIsNotReal_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a `BOOSE.CommandException` when the target variable is not a true BOOSE real variable, due to BOOSE runtime typing.

```
[TestMethod]
public void Execute_WhenVariableIsNotReal_ThrowsCommandException()
```

Remarks

Even if expression evaluation succeeds, BOOSE will reject [UpdateVariable\(string, double\)](#) unless the variable is typed as a real internally.

Set_StoresTrimmedExpression_DoesNotThrow()

Verifies that [Set\(StoredProgram, string\)](#) stores a trimmed expression and keeps a program reference.

```
[TestMethod]
public void Set_StoresTrimmedExpression_DoesNotThrow()
```

Class AppWhileTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [AppWhile](#). Validates condition evaluation, operator validation, and loop exit behaviour without relying on program runner.

```
[TestClass]
public class AppWhileTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← AppWhileTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

Compile_WhenOperatorInvalid_ThrowsParserException()

Ensures invalid comparison operators fail at compile time.

```
[TestMethod]
public void Compile_WhenOperatorInvalid_ThrowsParserException()
```

EvaluateCondition_WhenTrueCondition_ReturnsTrue()

Verifies EvaluateCondition returns true for valid expressions.

```
[TestMethod]
public void EvaluateCondition_WhenTrueCondition_ReturnsTrue()
```

Execute_WhenConditionFalse_SetsConditionFalse()

Ensures Execute evaluates condition as false.

```
[TestMethod]  
public void Execute_WhenConditionFalse_SetsConditionFalse()
```

Execute_WhenConditionTrue_SetsConditionTrue()

Ensures Execute evaluates condition as true.

```
[TestMethod]  
public void Execute_WhenConditionTrue_SetsConditionTrue()
```

Class CircleCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [CircleCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal radius values.

```
[TestClass]
public class CircleCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← CircleCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Remarks

The expression-evaluation path in [Execute\(\)](#) is intentionally not asserted here because the current implementation draws twice and contains a logic issue (it assigns `radius = radiusValue` even when the non-literal branch runs). These tests focus on the stable literal-path behavior and validation rules.

Methods

CheckParameters_WhenNotOne_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a `BOOSE.ParserException` when the parameter count is not exactly one.

```
[TestMethod]
public void CheckParameters_WhenNotOne_ThrowsParserException()
```

CheckParameters_WhenOne_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly one parameter is supplied.

```
[TestMethod]
public void CheckParameters_WhenOne_DoesNotThrow()
```

Compile_WhenLiteralRadiusIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when the radius is a literal integer less than zero.

```
[TestMethod]
public void Compile_WhenLiteralRadiusIsNegative_ThrowsParserException()
```

Compile_WhenLiteralRadiusNonNegative_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when the radius is a non-negative literal integer.

```
[TestMethod]
public void Compile_WhenLiteralRadiusNonNegative_DoesNotThrow()
```

Execute_WhenLiteralRadiusZero_DoesNotThrow()

Verifies that [Execute\(\)](#) can run when the literal radius is zero.

```
[TestMethod]
public void Execute_WhenLiteralRadiusZero_DoesNotThrow()
```

Remarks

The command does not explicitly forbid a radius of 0 at execution time in the current implementation. This test ensures the literal path does not throw for "0".

Execute_WhenLiteralRadius_SetsRadiusProperty()

Verifies that [Execute\(\)](#) sets [radius](#) to the literal radius when a valid literal is supplied.

```
[TestMethod]
```

```
public void Execute_WhenLiteralRadius_SetsRadiusProperty()
```

Set_WhenValid_SetsRadiusParam()

Verifies that [Set\(StoredProgram, string\)](#) parses a single parameter and assigns it to [radiusParam](#).

```
[TestMethod]  
public void Set_WhenValid_SetsRadiusParam()
```

Set_WhenWrongParameterCount_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is not exactly one.

```
[TestMethod]  
public void Set_WhenWrongParameterCount_ThrowsParserException()
```

Class CommandParserTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Unit tests for [CommandParser](#). Covers parsing of single-line commands (including special-cased "end if/while/for"), variable declarations (int/real), assignments, and loop/if structure validation.

```
[TestClass]
[DoNotParallelize]
public class CommandParserTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← CommandParserTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

ParseCommand_WhenEndFor_ReturnsAppEndFor()

Verifies special case: "end for" produces an [AppEndFor](#).

```
[TestMethod]
public void ParseCommand_WhenEndFor_ReturnsAppEndFor()
```

ParseCommand_WhenEndIf_ReturnsAppEndIf()

Verifies special case: "end if" produces an [AppEndIf](#).

```
[TestMethod]
public void ParseCommand_WhenEndIf_ReturnsAppEndIf()
```

ParseCommand_WhenEndWhile_ReturnsAppEndWhile()

Verifies special case: "end while" produces an [AppEndWhile](#).

```
[TestMethod]
public void ParseCommand_WhenEndWhile_ReturnsAppEndWhile()
```

ParseCommand_WhenLineEmpty_ThrowsParserException()

Verifies [ParseCommand\(string\)](#) throws when line is null/whitespace.

```
[TestMethod]
public void ParseCommand_WhenLineEmpty_ThrowsParserException()
```

ParseCommand_WhenMoveTo_ReturnsMoveToCommand()

Verifies a normal command is created via the factory and compiled, e.g. "moveto 10, 20".

```
[TestMethod]
public void ParseCommand_WhenMoveTo_ReturnsMoveToCommand()
```

ParseProgram_WhenAssignToExistingInt_CreatesAppIntCommand()

Verifies that an assignment to an existing int variable produces an [AppInt](#) command.

```
[TestMethod]
public void ParseProgram_WhenAssignToExistingInt_CreatesAppIntCommand()
```

ParseProgram_WhenAssignToUndeclaredVariable_ThrowsParserException()

Verifies that assigning to an undeclared variable throws BOOSE.ParserException.

```
[TestMethod]
public void ParseProgram_WhenAssignToUndeclaredVariable_ThrowsParserException()
```

ParseProgram_WhenElseWithoutIf_ThrowsParserException()

Verifies that "else" without a matching "if" throws a BOOSE.ParserException.

```
[TestMethod]
public void ParseProgram_WhenElseWithoutIf_ThrowsParserException()
```

ParseProgram_WhenEndIfWithoutIf_ThrowsParserException()

Verifies that "end if" without a matching "if" throws a BOOSE.ParserException.

```
[TestMethod]
public void ParseProgram_WhenEndIfWithoutIf_ThrowsParserException()
```

ParseProgram_WhenForClosed_SetsEndForLineNumber()

Verifies FOR/END FOR parses and that EndFor receives ForLineNumber.

```
[TestMethod]
public void ParseProgram_WhenForClosed_SetsEndForLineNumber()
```

ParseProgram_WhenForNotClosed_ThrowsParserException()

Verifies that an unclosed FOR is detected.

```
[TestMethod]
public void ParseProgram_WhenForNotClosed_ThrowsParserException()
```

ParseProgram_WhenIfClosed_DoesNotThrow()

Verifies that a simple IF/END IF program parses without throwing.

```
[TestMethod]
public void ParseProgram_WhenIfClosed_DoesNotThrow()
```

ParseProgram_WhenIfNotClosed_ThrowsParserException()

Verifies that an unclosed IF is detected.

```
[TestMethod]
public void ParseProgram_WhenIfNotClosed_ThrowsParserException()
```

ParseProgram_WhenIntDeclarationWithLiteral_AddsRuntimeAssignmentCommand()

Verifies that "int x = 5" creates the variable and adds an [AppInt](#) assignment command.

```
[TestMethod]
public void ParseProgram_WhenIntDeclarationWithLiteral_AddsRuntimeAssignmentCommand()
```

ParseProgram_WhenIntDeclaration_AddsIntVariable()

Verifies that "int x" creates an integer variable in the program.

```
[TestMethod]
public void ParseProgram_WhenIntDeclaration_AddsIntVariable()
```

ParseProgram_WhenNoCommandEntered_ThrowsParserException()

Verifies [ParseProgram\(string\)](#) throws when input is empty.

```
[TestMethod]
public void ParseProgram_WhenNoCommandEntered_ThrowsParserException()
```

ParseProgram_WhenRealDeclarationWithLiteral_AddsRuntimeAssignmentCommand()

Verifies that "real r = 2.5" creates the variable and adds an [AppReal](#) assignment command.

```
[TestMethod]
public void ParseProgram_WhenRealDeclarationWithLiteral_AddsRuntimeAssignmentCommand()
```

ParseProgram_WhenRealDeclaration_AddsRealVariable()

Verifies that "real r" creates a real variable in the program.

```
[TestMethod]
public void ParseProgram_WhenRealDeclaration_AddsRealVariable()
```

ParseProgram_WhenWhileClosed_SetsEndWhileLineNumber()

Verifies WHILE/END WHILE parses and that EndWhile receives WhileLineNumber.

```
[TestMethod]
public void ParseProgram_WhenWhileClosed_SetsEndWhileLineNumber()
```

ParseProgram_WhenWhileNotClosed_ThrowsParserException()

Verifies that an unclosed WHILE is detected.

```
[TestMethod]
public void ParseProgram_WhenWhileNotClosed_ThrowsParserException()
```

Setup()

Creates a fresh [CommandParser](#) for each test.

```
[TestInitialize]
```

```
public void Setup()
```

Class DrawToCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [DrawToCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

```
[TestClass]
public class DrawToCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← DrawToCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotTwo_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when the parameter count is not exactly two.

```
[TestMethod]
public void CheckParameters_WhenNotTwo_ThrowsParserException()
```

CheckParameters_WhenTwo_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly two parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenTwo_DoesNotThrow()
```

Compile_WhenLiteralCoordinatesNonNegative_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when both literal coordinates are non-negative.

```
[TestMethod]
public void Compile_WhenLiteralCoordinatesNonNegative_DoesNotThrow()
```

Compile_WhenLiteralXIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal X coordinate is negative.

```
[TestMethod]
public void Compile_WhenLiteralXIsNegative_ThrowsParserException()
```

Compile_WhenLiteralYIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal Y coordinate is negative.

```
[TestMethod]
public void Compile_WhenLiteralYIsNegative_ThrowsParserException()
```

Execute_WhenExpressionEvaluatesNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when an evaluated coordinate is negative.

```
[TestMethod]
public void Execute_WhenExpressionEvaluatesNegative_ThrowsCommandException()
```

Execute_WhenLiteralCoordinatesValid_UpdatesXAndYFields()

Verifies that [Execute\(\)](#) updates the command's stored integer fields ([x](#) and [y](#)) when valid literal coordinates are supplied.

```
[TestMethod]
public void Execute_WhenLiteralCoordinatesValid_UpdatesXAndYFields()
```

Execute_WhenLiteralXIsNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the resolved X coordinate is negative (literal path).

```
[TestMethod]
public void Execute_WhenLiteralXIsNegative_ThrowsCommandException()
```

Execute_WhenLiteralYIsNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the resolved Y coordinate is negative (literal path).

```
[TestMethod]
public void Execute_WhenLiteralYIsNegative_ThrowsCommandException()
```

Execute_WhenParametersAreExpressions_EvaluatesAndDraws()

Verifies that [Execute\(\)](#) evaluates non-literal parameters using [EvaluateExpressionWithString\(string\)](#)[↗] and draws to the resolved coordinates.

```
[TestMethod]
public void Execute_WhenParametersAreExpressions_EvaluatesAndDraws()
```

Remarks

In this BOOSE build, "+" may be treated as concatenation (e.g., "10+5" may become "105"), so this test uses subtraction which is observed to evaluate numerically.

Set_WhenValid_SetsXParamAndYParam()

Verifies that [Set\(StoredProgram, string\)](#) parses two parameters and assigns them to [xParam](#) and [yParam](#).

```
[TestMethod]  
public void Set_WhenValid_SetsXParamAndYParam()
```

Set_WhenWrongParameterCount_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is not exactly two.

```
[TestMethod]  
public void Set_WhenWrongParameterCount_ThrowsParserException()
```

Class MoveToCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [MoveToCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

```
[TestClass]
public class MoveToCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#)  ← MoveToCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotTwo_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a [BOOSE.ParserException](#) when the parameter count is not exactly two.

```
[TestMethod]
public void CheckParameters_WhenNotTwo_ThrowsParserException()
```

CheckParameters_WhenTwo_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly two parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenTwo_DoesNotThrow()
```

Compile_WhenLiteralCoordinatesNonNegative_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when both literal coordinates are non-negative.

```
[TestMethod]
public void Compile_WhenLiteralCoordinatesNonNegative_DoesNotThrow()
```

Compile_WhenLiteralXIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal X coordinate is negative.

```
[TestMethod]
public void Compile_WhenLiteralXIsNegative_ThrowsParserException()
```

Compile_WhenLiteralYIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal Y coordinate is negative.

```
[TestMethod]
public void Compile_WhenLiteralYIsNegative_ThrowsParserException()
```

Execute_WhenExpressionEvaluatesNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when an evaluated coordinate is negative.

```
[TestMethod]
public void Execute_WhenExpressionEvaluatesNegative_ThrowsCommandException()
```

Execute_WhenLiteralCoordinatesValid_UpdatesXAndYFields()

Verifies that [Execute\(\)](#) updates the command's stored integer fields ([x](#) and [y](#)) when valid literal coordinates are supplied.

```
[TestMethod]
public void Execute_WhenLiteralCoordinatesValid_UpdatesXAndYFields()
```

Execute_WhenLiteralXIsNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the resolved X coordinate is negative (literal path).

```
[TestMethod]
public void Execute_WhenLiteralXIsNegative_ThrowsCommandException()
```

Execute_WhenLiteralYIsNegative_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the resolved Y coordinate is negative (literal path).

```
[TestMethod]
public void Execute_WhenLiteralYIsNegative_ThrowsCommandException()
```

Execute_WhenParametersAreExpressions_EvaluatesAndMoves()

Verifies that [Execute\(\)](#) evaluates non-literal parameters using [EvaluateExpressionWithString\(string\)](#)[↗] and moves to the resolved coordinates.

```
[TestMethod]
public void Execute_WhenParametersAreExpressions_EvaluatesAndMoves()
```

Remarks

In this BOOSE build, "+" may be treated as concatenation (e.g., "10+5" may become "105"), so this test uses subtraction which is observed to evaluate numerically.

Set_WhenValid_SetsXParamAndYParam()

Verifies that [Set\(StoredProgram, string\)](#) parses the two parameters and assigns them to [xParam](#) and [yParam](#).

```
[TestMethod]  
public void Set_WhenValid_SetsXParamAndYParam()
```

Set_WhenWrongParameterCount_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is not exactly two.

```
[TestMethod]  
public void Set_WhenWrongParameterCount_ThrowsParserException()
```

Class PenCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [PenCommand](#) class. Tests cover parameter validation, compile-time validation for literal RGB values, and execution-time behavior including runtime range checking.

```
[TestClass]
public class PenCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← PenCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotThree_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when the parameter count is not exactly three.

```
[TestMethod]
public void CheckParameters_WhenNotThree_ThrowsParserException()
```

CheckParameters_WhenThree_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly three parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenThree_DoesNotThrow()
```

Compile_WhenLiteralBlueOutOfRange_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal blue value is outside the inclusive range 0..255.

```
[TestMethod]
```

```
public void Compile_WhenLiteralBlueOutOfRange_ThrowsParserException()
```

Compile_WhenLiteralGreenOutOfRange_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal green value is outside the inclusive range 0..255.

```
[TestMethod]
```

```
public void Compile_WhenLiteralGreenOutOfRange_ThrowsParserException()
```

Compile_WhenLiteralRedOutOfRange_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal red value is outside the inclusive range 0..255.

```
[TestMethod]
```

```
public void Compile_WhenLiteralRedOutOfRange_ThrowsParserException()
```

Compile_WhenLiteralRgbInRange_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when all literal RGB values are within the inclusive range 0..255.

```
[TestMethod]
```

```
public void Compile_WhenLiteralRgbInRange_DoesNotThrow()
```

Execute_WhenBlueOutOfRange_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the evaluated blue value is outside the inclusive range 0..255.

```
[TestMethod]
public void Execute_WhenBlueOutOfRange_ThrowsCommandException()
```

Execute_WhenGreenOutOfRange_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the evaluated green value is outside the inclusive range 0..255.

```
[TestMethod]
public void Execute_WhenGreenOutOfRange_ThrowsCommandException()
```

Execute_WhenRedOutOfRange_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the evaluated red value is outside the inclusive range 0..255.

```
[TestMethod]
public void Execute_WhenRedOutOfRange_ThrowsCommandException()
```

Execute_WhenRgbInRange_SetsCanvasPenColour()

Verifies that [Execute\(\)](#) sets the canvas pen colour when all evaluated values are within the inclusive range 0..255.

```
[TestMethod]
public void Execute_WhenRgbInRange_SetsCanvasPenColour()
```

Set_WhenValid_SetsRgbParameterStrings()

Verifies that [Set\(StoredProgram, string\)](#) parses and stores the RGB parameters into [redParam](#), [greenParam](#), and [blueParam](#).

```
[TestMethod]  
public void Set_WhenValid_SetsRgbParameterStrings()
```

Set_WhenWrongParameterCount_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is not exactly three.

```
[TestMethod]  
public void Set_WhenWrongParameterCount_ThrowsParserException()
```

Class RectangleCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [RectangleCommand](#) class. Tests cover parameter validation, compile-time validation for literal inputs, and execution-time behavior for literal and evaluated parameters.

```
[TestClass]
public class RectangleCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#) ← RectangleCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotTwo_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws BOOSE.ParserException when the parameter count is not exactly two.

```
[TestMethod]
public void CheckParameters_WhenNotTwo_ThrowsParserException()
```

CheckParameters_WhenTwo_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly two parameters are supplied.

```
[TestMethod]
public void CheckParameters_WhenTwo_DoesNotThrow()
```

Compile_WhenLiteralHeightIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal height is negative.

```
[TestMethod]
public void Compile_WhenLiteralHeightIsNegative_ThrowsParserException()
```

Compile_WhenLiteralWidthAndHeightNonNegative_DoesNotThrow()

Verifies that [Compile\(\)](#) does not throw when literal width and height are non-negative.

```
[TestMethod]
public void Compile_WhenLiteralWidthAndHeightNonNegative_DoesNotThrow()
```

Compile_WhenLiteralWidthIsNegative_ThrowsParserException()

Verifies that [Compile\(\)](#) throws a BOOSE.ParserException when a literal width is negative.

```
[TestMethod]
public void Compile_WhenLiteralWidthIsNegative_ThrowsParserException()
```

Execute_WhenHeightExpressionEvaluatesToZero_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when an evaluated height expression results in a value less than one.

```
[TestMethod]
public void Execute_WhenHeightExpressionEvaluatesToZero_ThrowsCommandException()
```

Execute_WhenLiteralHeightIsZero_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the literal height is less than one.

```
[TestMethod]
public void Execute_WhenLiteralHeightIsZero_ThrowsCommandException()
```

Execute_WhenLiteralParametersValid_UpdatesWidthAndHeightFields()

Verifies that [Execute\(\)](#) stores the resolved width and height in the command fields when valid literal parameters are supplied.

```
[TestMethod]
public void Execute_WhenLiteralParametersValid_UpdatesWidthAndHeightFields()
```

Execute_WhenLiteralWidthIsZero_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when the literal width is less than one.

```
[TestMethod]
public void Execute_WhenLiteralWidthIsZero_ThrowsCommandException()
```

Execute_WhenParametersAreExpressions_EvaluatesAndUpdatesFields()

Verifies that [Execute\(\)](#) evaluates non-literal parameters using [EvaluateExpressionWithString\(string\)](#)[↗] and uses the resolved values.

```
[TestMethod]
public void Execute_WhenParametersAreExpressions_EvaluatesAndUpdatesFields()
```

Remarks

In this BOOSE build, "+" may be treated as concatenation (e.g., "10+5" may become "105"), so this test uses subtraction which is observed to evaluate numerically.

Execute_WhenWidthExpressionEvaluatesToZero_ThrowsCommandException()

Verifies that [Execute\(\)](#) throws a BOOSE.CommandException when an evaluated width expression results in a value less than one.

```
[TestMethod]
public void Execute_WhenWidthExpressionEvaluatesToZero_ThrowsCommandException()
```

Set_WhenValid_SetsWidthParamAndHeightParam()

Verifies that [Set\(StoredProgram, string\)](#) parses width and height and assigns them to [widthParam](#) and [heightParam](#).

```
[TestMethod]
public void Set_WhenValid_SetsWidthParamAndHeightParam()
```

Set_WhenWrongParameterCount_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the parameter count is not exactly two.

```
[TestMethod]
public void Set_WhenWrongParameterCount_ThrowsParserException()
```

Class TestBase

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Provides common setup for BOOSE command unit tests, including a BOOSE.StoredProgram created with a headless BOOSE.ICanvas.

```
public abstract class TestBase
```

Inheritance

[object](#)  ← TestBase

Derived

[AppElseTests](#), [AppEndForTests](#), [AppEndIfTests](#), [AppEndWhileTests](#), [AppForTests](#), [AppIfTests](#), [AppIntTests](#), [AppPeekTests](#), [AppPokeTests](#), [AppRealTests](#), [AppWhileTests](#), [CircleCommandTests](#), [CommandParserTests](#), [DrawToCommandTests](#), [MoveToCommandTests](#), [PenCommandTests](#), [RectangleCommandTests](#), [WriteCommandTests](#)

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Properties

Canvas

A headless canvas instance used to satisfy BOOSE.StoredProgram's constructor requirement.

```
protected ICanvas Canvas { get; }
```

Property Value

ICanvas

Program

A program instance created fresh for each test to avoid shared state.

```
protected StoredProgram Program { get; }
```

Property Value

StoredProgram

Methods

BaseSetup()

Creates a new BOOSE.StoredProgram and BOOSE.ICanvas before each test.

```
[TestInitialize]  
public void BaseSetup()
```

Class WriteCommandTests

Namespace: [BOOSEUnitTesting](#)

Assembly: BOOSEUnitTesting.dll

Contains unit tests for the [WriteCommand](#) class. Tests cover parameter validation and execution behavior for both evaluated text and literal (fallback) text.

```
[TestClass]
public class WriteCommandTests : TestBase
```

Inheritance

[object](#)  ← [TestBase](#)  ← WriteCommandTests

Inherited Members

[TestBase.Program](#) , [TestBase.Canvas](#) , [TestBase.BaseSetup\(\)](#) , [object.Equals\(object\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

CheckParameters_WhenNotOne_ThrowsParserException()

Verifies that [CheckParameters\(string\[\]\)](#) throws a BOOSE.ParserException when the parameter count is not exactly one.

```
[TestMethod]
public void CheckParameters_WhenNotOne_ThrowsParserException()
```

CheckParameters_WhenOne_DoesNotThrow()

Verifies that [CheckParameters\(string\[\]\)](#) does not throw when exactly one parameter is supplied.

```
[TestMethod]
public void CheckParameters_WhenOne_DoesNotThrow()
```

Compile_DoesNotThrow()

Verifies that calling [Compile\(\)](#) does not throw, as compilation is currently not implemented.

```
[TestMethod]
public void Compile_DoesNotThrow()
```

Execute_WhenEvaluationFails_FallsBackToLiteralText()

Verifies that [Execute\(\)](#) falls back to treating the parameter as a literal string when evaluation throws.

```
[TestMethod]
public void Execute_WhenEvaluationFails_FallsBackToLiteralText()
```

Execute_WhenEvaluationSucceeds_SetsTextToEvaluatedValue()

Verifies that [Execute\(\)](#) sets [text](#) to the evaluated value when [EvaluateExpressionWithString\(string\)](#)  succeeds.

```
[TestMethod]
public void Execute_WhenEvaluationSucceeds_SetsTextToEvaluatedValue()
```

Remarks

This test uses a numeric-like string because in this BOOSE build the evaluator is known to return a value for such inputs.

Set_TrimsWhitespace()

Verifies that leading and trailing whitespace is trimmed by [Set\(StoredProgram, string\)](#) before being stored as [textParam](#).

```
[TestMethod]
public void Set_TrimsWhitespace()
```

Set_WhenEmptyString_ThrowsParserException()

Verifies that [Set\(StoredProgram, string\)](#) throws a BOOSE.ParserException when the supplied text is empty (treated as zero parameters).

```
[TestMethod]  
public void Set_WhenEmptyString_ThrowsParserException()
```

Set_WhenValid_SetsTextParam()

Verifies that [Set\(StoredProgram, string\)](#) stores the raw parameter string as [textParam](#).

```
[TestMethod]  
public void Set_WhenValid_SetsTextParam()
```

Namespace BOOSE_App

Classes

[AppArray](#)

Array command class which creates an array variable for BOOSE.StoredProgram. This class creates both integer and real arrays and bypasses BOOSE restrictions.

[AppCanvas](#)

Class used for a drawable canvas via underlying bitmap. Supports basic drawing operations which are built upon in other classes.

[AppCommandFactory](#)

Factory class responsible for creating command objects for BOOSE application. Commands are linked to [AppCanvas](#) allowing drawing operations to be used avoiding their inherited restrictions.

[AppElse](#)

Else command class which executes code if a false value is passed into the stack by the if command. Bypasses BOOSE restrictions.

[AppEndFor](#)

End for command class which marks the end of a for loop. Determines whether loop should repeat or end, bypasses BOOSE restrictions.

[AppEndIf](#)

End if command class used to mark the end of an if block and updates runtimestack. Bypasses BOOSE restrictions.

[AppEndWhile](#)

End command class which marks the end of a while loop. Bypasses BOOSE restrictions.

[AppFor](#)

For command class which executes a loop by iterating an integer variable from a start value to an end value using an optional step.

[AppFor.ForRuntime](#)

Maintains a stack of active for commands.

[AppIf](#)

If command class which evaluates two expressions to set a boolean condition. Command execution is determined based on the boolean result. Bypasses BOOSE restrictions.

[AppIf.IfRuntime](#)

Maintains a stack of active if commands.

[AppInt](#)

Command class for handling integer variable assignments. Replaces BOOSE.Int functionality for runtime expression evaluation.

[AppPeek](#)

Peek command class which reads a value from an array and assigns it to an existing variable. Bypasses BOOSE restrictions.

[AppPoke](#)

Poke command which writes a value into an array element. Bypasses BOOSE restrictions.

[AppReal](#)

Command class for handling real (double) variable assignments. Replaces BOOSE.Real functionality for runtime expression evaluation.

[AppWhile](#)

While command class which evaluates two expressions to set a boolean condition. Command execution is repeatedly performed while the condition remains true.

[AppWhile.WhileRuntime](#)

Maintains a stack of active while commands.

[CircleCommand](#)

Command class for drawing circles onto canvas at current pen position. Bypasses BOOSE restrictions. Uses BOOSE.ICommand interface.

[CommandParser](#)

Parses user input into valid BOOSE commands. Works with command factory to create command objects and stores them in BOOSE.StoredProgram for execution.

[DrawToCommand](#)

Command class for drawing a line on canvas from current pen position.

[Form1](#)

[Form1](#) operates as the main class for the application. It provides a windows form GUI to store and execute user commands, which interact with the drawable canvas.

[MoveToCommand](#)

Command class for moving pen on canvas without drawing.

[PenCommand](#)

Command class used for setting the pen colour used for drawing on canvas.

[RectangleCommand](#)

Command class for drawing rectangles onto canvas at current pen position. Bypasses BOOSE restrictions.

[WriteCommand](#)

Command class for drawing text onto canvas at current pen position. Uses BOOSE.ICommand interface.

Class AppArray

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Array command class which creates an array variable for BOOSE.StoredProgram. This class creates both integer and real arrays and bypasses BOOSE restrictions.

```
public class AppArray : Evaluation, ICommand
```

Inheritance

[object](#)  ← Command ← Evaluation ← AppArray

Implements

ICommand

Inherited Members

Evaluation.expression , Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value , [Evaluation.ProcessExpression\(string\)](#)  , Evaluation.Expression , Evaluation.VarName , Evaluation.Value , Evaluation.Local , Command.IsDouble , Command.parameterList , Command.parameters , Command.paramsint , [Command.ProcessParameters\(string\)](#)  , Command.ToString() , Command.Program , Command.Name , Command.ParameterList , Command.Parameters , Command.Paramsint , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Fields

arrayType

```
public string arrayType
```

Field Value

[string](#) 

program

```
public StoredProgram program
```

Field Value

StoredProgram

size

```
public int size
```

Field Value

[int](#)

Properties

ArrayType

```
public string ArrayType { get; }
```

Property Value

[string](#)

IntArray

```
public int[] IntArray { get; set; }
```

Property Value

[int](#) []

RealArray

```
public double[] RealArray { get; set; }
```

Property Value

[double](#) []

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#) []

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not three parameters stored as expected.

Compile()

Validates at compile time the array declaration is valid and stores the array in BOOSE.StoredProgram so it can be referenced by [AppPeek](#) and [AppPoke](#) before execution.

```
public void Compile()
```

Exceptions

ParserException

Thrown when array type isn't int or real or when the size declared is invalid.

Execute()

```
public void Execute()
```

GetInt(int)

Returns an integer element from the array.

```
public int GetInt(int index)
```

Parameters

index [int](#)

The index of the element to be returned.

Returns

[int](#)

The value stored at specified index.

Exceptions

CommandException

Thrown if the array is not an integer array, is not allocated, or the index is out of bounds.

GetReal(int)

Returns a real element from the array.

```
public double GetReal(int index)
```

Parameters

index [int](#)

The index of the element to be returned.

Returns

[double](#)

The value stored at specified index.

Exceptions

CommandException

Thrown if the array is not an integer array, is not allocated, or the index is out of bounds.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

SetInt(int, int)

Sets an integer value at the specified index in the array.

```
public void SetInt(int val, int index)
```

Parameters

val [int](#)

The int value to be set.

index [int](#)

The index of the array where the value will be set

Exceptions

CommandException

Thrown if the array is not an integer array, is not allocated, or the index is out of bounds.

SetReal(double, int)

Sets a real value at the specified index in the array.

```
public void SetReal(double val, int index)
```

Parameters

val [double](#)

The real value to be set.

index [int](#)

The index of the array where the value will be set.

Exceptions

CommandException

Thrown if the array is not a real array, is not allocated, or the index is out of bounds.

Class AppCanvas

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Class used for a drawable canvas via underlying bitmap. Supports basic drawing operations which are built upon in other classes.

```
public class AppCanvas : ICanvas
```

Inheritance

[object](#) ← AppCanvas

Implements

ICanvas

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

AppCanvas(int, int)

Constructor for [AppCanvas](#) which sets up size parameters, pen properties and bitmap to canvas.

```
public AppCanvas(int width, int height)
```

Parameters

width [int](#)

The width of the canvas.

height [int](#)

The height of the canvas.

Fields

background_colour

`protected Color background_colour`

Field Value

[Color](#)

height

`public int height`

Field Value

[int](#)

width

`public int width`

Field Value

[int](#)

xpos

`public int xpos`

Field Value

[int](#)

ypos

```
public int ypos
```

Field Value

[int](#)

Properties

PenColour

```
public virtual object PenColour { get; set; }
```

Property Value

[object](#)

Xpos

Returns X position of pen on canvas.

Sets X position of pen on canvas. Catches BOOSE.CanvasException if value isn't within canvas boundaries.

```
public int Xpos { get; set; }
```

Property Value

[int](#)

Ypos

Returns Y position of pen on canvas.

Sets Y position of pen on canvas. Catches BOOSE.CanvasException if value isn't within canvas boundaries.

```
public int Ypos { get; set; }
```

Property Value

[int](#)

Methods

Circle(int, bool)

Draws a circle at current pen position.

```
public void Circle(int radius, bool filled)
```

Parameters

radius [int](#)

Radius of circle to be drawn.

filled [bool](#)

Determines whether circle is filled or not. Currently not implemented.

Remarks

Updates the pen position after drawing.

Clear()

Clears canvas.

```
public virtual void Clear()
```

DrawTo(int, int)

Draws a line on canvas from current pen position to desired x and y co-ordinates. Throws BOOSE.CanvasException if parameter isn't within canvas boundaries.

```
public void DrawTo(int x, int y)
```

Parameters

x [int](#)

X co-ordinate value of Pen.

y [int](#)

Y co-ordinate value of Pen.

Remarks

Updates the pen position to the specified co-ordinates after drawing.

MoveTo(int, int)

Moves pen to desired x and y co-ordinates without drawing. Throws BOOSE.CanvasException if parameter isn't within canvas boundaries.

```
public void MoveTo(int x, int y)
```

Parameters

x [int](#)

X co-ordinate value of Pen.

y [int](#)

Y co-ordinate value of Pen.

Rect(int, int, bool)

Draws rectangle at the current pen position.

```
public void Rect(int width, int height, bool filled)
```

Parameters

width [int](#)

Width of rectangle.

height [int](#)

Height of rectangle.

filled [bool](#)

Determines whether rectangle is filled or not. Currently not implemented.

Remarks

Updates the pen position after drawing.

Reset()

Resets pen position to default values (0,0).

```
public virtual void Reset()
```

Set(int, int)

Resets graphic context for existing bitmap.

```
public virtual void Set(int width, int height)
```

Parameters

width [int](#)

Width of canvas.

height [int](#)

Width of bitmap.

SetColour(int, int, int)

Sets colour of pen via RGB values. Throws BOOSE.CanvasException if a value is above 255.

```
public void SetColour(int red, int green, int blue)
```

Parameters

red [int](#)

Red colour value.

green [int](#)

Green colour value.

blue [int](#)

Blue colour value.

Tri(int, int)

Draws triangle at current pen position with left corner as base point.

```
public virtual void Tri(int width, int height)
```

Parameters

width [int](#)

Width of triangle.

height [int](#)

Height of triangle.

Remarks

Triangle is drawn with three points the current position, the top point and the base right point. Updates pen position after drawing.

WriteText(string)

Draws text at current pen position.

```
public virtual void WriteText(string text)
```

Parameters

text [string](#) 

String to be drawn on canvas.

Remarks

Updates the pen positions after drawing.

draw(Graphics)

Method had to be implemented due to interface. But is not in use as of right now.

```
public void draw(Graphics g)
```

Parameters

g [Graphics](#) 

Exceptions

[NotImplementedException](#) 

getBitmap()

Returns bitmap.

```
public virtual object getBitmap()
```

Returns

[object](#)

Bitmap which is used for drawing operations.

Class AppCommandFactory

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Factory class responsible for creating command objects for BOOSE application. Commands are linked to [AppCanvas](#) allowing drawing operations to be used avoiding their inherited restrictions.

```
public class AppCommandFactory : CommandFactory, ICommandFactory
```

Inheritance

[object](#)  ← CommandFactory ← AppCommandFactory

Implements

ICommandFactory

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

AppCommandFactory(AppCanvas)

Constructor for [AppCommandFactory](#) setting up canvas for commands.

```
public AppCommandFactory(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

The canvas which commands will operate on.

Fields

canvas

Canvas object that commands use.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

Methods

MakeCommand(string)

Creates command objects based on provided name.

```
public override ICommand MakeCommand(string commandType)
```

Parameters

commandType [string](#) 

The name of the command to but be created.

Returns

ICommand

Command for execution.

Exceptions

FactoryException

Thrown when the command entered doesn't exist.

Class AppElse

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Else command class which executes code if a false value is passed into the stack by the if command.
Bypasses BOOSE restrictions.

```
public class AppElse : ICommand
```

Inheritance

[object](#) ← AppElse

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not zero parameters stored as expected.

Compile()

Method not implemented.

```
public void Compile()
```

Execute()

Executes else command by determining whether the associated if block has already been executed.

```
public void Execute()
```

Remarks

If the corresponding if condition is true execution skips directly to the command following end if. Otherwise, execution continues into the else block.

Exceptions

CommandException

Thrown when there is no matching **if** statement on the runtime stack.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppEndFor

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

End for command class which marks the end of a for loop. Determines whether loop should repeat or end, bypasses BOOSE restrictions.

```
public class AppEndFor : ICommand
```

Inheritance

[object](#) ← AppEndFor

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Properties

ForLineNumber

Gets or sets the line number of the corresponding for command. This value is used to jump back to the start of the loop when additional iterations are required.

```
public int ForLineNumber { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not zero parameters stored as expected.

Compile()

Method not implemented.

```
public void Compile()
```

Execute()

Executes end for command by determining whether the associated for loop should continue or terminate.

```
public void Execute()
```

Exceptions

CommandException

Thrown when there is no matching for command on the runtime stack.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppEndIf

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

End if command class used to mark the end of an if block and updates runtimestack. Bypasses BOOSE restrictions.

```
public class AppEndIf : ICommand
```

Inheritance

[object](#) ← AppEndIf

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Fields

program

```
public StoredProgram program
```

Field Value

StoredProgram

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not zero parameters stored as expected.

Compile()

Method not implemented.

```
public void Compile()
```

Execute()

Executes the command by removing the most recent if command from the runtime stack.

```
public void Execute()
```

Exceptions

CommandException

Thrown when there is no matching if command on the stack.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppEndWhile

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

End command class which marks the end of a while loop. Bypasses BOOSE restrictions.

```
public class AppEndWhile : ICommand
```

Inheritance

[object](#) ← AppEndWhile

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Properties

WhileLineNumber

Gets or sets the line number of the corresponding while command . This value is used to jump back to the start of the loop when the loop condition is true.

```
public int WhileLineNumber { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not zero parameters stored as expected.

Compile()

method not implemented

```
public void Compile()
```

Execute()

Executes end while command by either looping back to the corresponding while command or exiting the loop.

```
public void Execute()
```

Exceptions

CommandException

Thrown when there is no matching `while` statement on the runtime stack.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppFor

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

For command class which executes a loop by iterating an integer variable from a start value to an end value using an optional step.

```
public class AppFor : ICommand
```

Inheritance

[object](#) ← AppFor

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Fields

EndValue

Evaluated end value of the loop.

```
public int EndValue
```

Field Value

[int](#)

StartValue

Evaluated start value of the loop.

```
public int StartValue
```

Field Value

[int](#)

Started

Indicates whether the loop has started execution. Used to differentiate the first iteration from later iterations.

```
public bool Started
```

Field Value

[bool](#)

StepValue

Evaluated step value of the loop.

```
public int StepValue
```

Field Value

[int](#)

currentValue

Current value of the loop control variable.

```
public int currentValue
```

Field Value

[int](#)

endValueStr

String representation of the end value expression.

```
public string endValueStr
```

Field Value

[string](#)

program

```
public StoredProgram program
```

Field Value

StoredProgram

startValueStr

```
public string startValueStr
```

Field Value

[string](#)

stepValueStr

String representation of the step value expression.

```
public string stepValueStr
```

Field Value

[string](#)

variableName

Name of the loop control variable.

```
public string variableName
```

Field Value

[string](#)

Properties

EndLineNumber

Gets or sets the line number of the corresponding **end** **for** command.

```
public int EndLineNumber { get; set; }
```

Property Value

[int](#)

LineNumber

Gets or sets the line number of this **for** command.

```
public int LineNumber { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not three parameters stored as expected.

Compile()

Compiles for command by validating loop variable type.

```
public void Compile()
```

Exceptions

ParserException

Thrown when an invalid operator is supplied.

Execute()

Executes for loop by evaluating bounds, updating the loop variable, and determining whether to continue or exit.

```
public void Execute()
```

Exceptions

CommandException

Thrown when the step value evaluates to zero.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppFor.ForRuntime

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Maintains a stack of active for commands.

```
public static class AppFor.ForRuntime
```

Inheritance

[object](#)  ← AppFor.ForRuntime

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Stack

```
public static Stack<AppFor> Stack { get; }
```

Property Value

[Stack](#)  <[AppFor](#)>

Class AppIf

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

If command class which evaluates two expressions to set a boolean condition. Command execution is determined based on the boolean result. Bypasses BOOSE restrictions.

```
public class AppIf : ConditionalCommand, ICommand
```

Inheritance

[object](#)  ← [Command](#) ← [Evaluation](#) ← [Boolean](#) ← [ConditionalCommand](#) ← [AppIf](#)

Implements

ICommand

Inherited Members

[ConditionalCommand.EndLineNumber](#) , [ConditionalCommand.EndLineNumber](#) ,
[ConditionalCommand.Condition](#) , [ConditionalCommand.LineNumber](#) , [ConditionalCommand.CondType](#) ,
[ConditionalCommand.ReturnLineNumber](#) , [Boolean.BoolValue](#) , [Evaluation.expression](#) ,
[Evaluation.evaluatedExpression](#) , [Evaluation.varName](#) , [Evaluation.value](#) ,
[Evaluation.ProcessExpression\(string\)](#)  , [Evaluation.Expression](#) , [Evaluation.VarName](#) , [Evaluation.Value](#) ,
[Evaluation.Local](#) , [Command.IsDouble](#) , [Command.parameterList](#) , [Command.parameters](#) ,
[Command.paramsint](#) , [Command.ProcessParameters\(string\)](#)  , [Command.ToString\(\)](#) ,
[Command.Program](#) , [Command.Name](#) , [Command.ParameterList](#) , [Command.Parameters](#) ,
[Command.Paramsint](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Fields

conditionResult

Stores the result of the evaluated condition.

```
public bool conditionResult
```

Field Value

[bool](#)

lhs

Left-hand side of the conditional expression.

```
public string lhs
```

Field Value

[string](#)

op

Comparison operator used in the condition.

```
public string op
```

Field Value

[string](#)

program

```
public StoredProgram program
```

Field Value

StoredProgram

rhs

Right-hand side of the conditional expression.

```
public string rhs
```

Field Value

[string](#)

Properties

ElseLineNumber

Gets or sets the line number of the corresponding else command.

```
public int ElseLineNumber { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public override void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not three parameters stored as expected.

Compile()

Compiles if command by validating the comparison operator.

```
public override void Compile()
```

Exceptions

ParserException

Thrown when an invalid operator is supplied.

EvaluateCondition()

Evaluates the conditional expression by resolving operators and applying the comparison operator.

```
public bool EvaluateCondition()
```

Returns

[bool](#)

true if the condition evaluates to true; otherwise **false**.

Exceptions

CommandException

Thrown when an invalid operator is encountered at runtime.

Execute()

Executes if command by evaluating its condition and adjusting the program counter if the condition is false.

```
public override void Execute()
```

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public override void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class Applf.IfRuntime

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Maintains a stack of active if commands.

```
public static class AppIf.IfRuntime
```

Inheritance

[object](#)  ← Applf.IfRuntime

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Properties

Stack

```
public static Stack<AppIf> Stack { get; }
```

Property Value

[Stack](#)  <[AppIf](#)>

Class AppInt

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for handling integer variable assignments. Replaces BOOSE.Int functionality for runtime expression evaluation.

```
public class AppInt : ICommand
```

Inheritance

[object](#)  ← AppInt

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

program

```
public StoredProgram program
```

Field Value

StoredProgram

Methods

CheckParameters(string[])

Checks parameters for assignment command.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Parameter array.

Exceptions

ParserException

Thrown if parameter count is incorrect.

Compile()

Compiles the assignment command. Validates that the variable exists but doesn't evaluate the expression.

```
public void Compile()
```

Execute()

Executes the assignment command at runtime. Evaluates the expression and updates the variable value.

```
public void Execute()
```

Set(StoredProgram, string)

Sets command parameters.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference to the stored program.

Params [string](#)

The expression to evaluate and assign.

SetVariableName(string)

Sets the variable name for this assignment.

```
public void SetVariableName(string varName)
```

Parameters

varName [string](#)

Name of the variable to assign to.

Class AppPeek

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Peek command class which reads a value from an array and assigns it to an existing variable. Bypasses BOOSE restrictions.

```
public class AppPeek : ICommand
```

Inheritance

[object](#) ← AppPeek

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not four parameters stored as expected.

Compile()

Performs compile time validation to ensure the target variable and array exist.

```
public void Compile()
```

Exceptions

ParserException

Thrown when the target variable does not exist or the array does not exist.

Execute()

Executes peek command reading the array value at the specified index and writing the value into the target variable.

```
public void Execute()
```

Exceptions

CommandException

Thrown if the referenced variable is not an array.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppPoke

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Poke command which writes a value into an array element. Bypasses BOOSE restrictions.

```
public class AppPoke : ICommand
```

Inheritance

[object](#) ← AppPoke

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not four parameters stored as expected.

Compile()

Performs compile time validation to ensure the target array exists.

```
public void Compile()
```

Exceptions

ParserException

Thrown when the target array does not exist.

Execute()

Executes poke command evaluating index and value expressions storing the value in the array at the specified index.

```
public void Execute()
```

Exceptions

CommandException

Thrown if the referenced variable is not an array (i.e., not an [AppArray](#)).

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppReal

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for handling real (double) variable assignments. Replaces BOOSE.Real functionality for runtime expression evaluation.

```
public class AppReal : ICommand
```

Inheritance

[object](#)  ← AppReal

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

program

```
public StoredProgram program
```

Field Value

StoredProgram

Methods

CheckParameters(string[])

Checks parameters for assignment command.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Parameter array.

Exceptions

ParserException

Thrown if parameter count is incorrect.

Compile()

Compiles the assignment command, validating that the variable exists but doesn't evaluate the expression.

```
public void Compile()
```

Execute()

Evaluates the expression and updates the variable value.

```
public void Execute()
```

Set(StoredProgram, string)

Sets command parameters.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference to the stored program.

Params [string](#)

The expression to evaluate and assign.

SetVariableName(string)

Sets the variable name for this assignment.

```
public void SetVariableName(string varName)
```

Parameters

varName [string](#)

Name of the variable to assign to.

Class AppWhile

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

While command class which evaluates two expressions to set a boolean condition. Command execution is repeatedly performed while the condition remains true.

```
public class AppWhile : ConditionalCommand, ICommand
```

Inheritance

[object](#)  ← Command ← Evaluation ← Boolean ← ConditionalCommand ← AppWhile

Implements

ICommand

Inherited Members

ConditionalCommand.endLineNumber , ConditionalCommand.EndLineNumber ,
ConditionalCommand.Condition , ConditionalCommand.LineNumber , ConditionalCommand.CondType ,
ConditionalCommand.ReturnLineNumber , Boolean.BoolValue , Evaluation.expression ,
Evaluation.evaluatedExpression , Evaluation.varName , Evaluation.value ,
[Evaluation.ProcessExpression\(string\)](#)  , Evaluation.Expression , Evaluation.VarName , Evaluation.Value ,
Evaluation.Local , Command.IsDouble , Command.parameterList , Command.parameters ,
Command.paramsint , [Command.ProcessParameters\(string\)](#)  , Command.ToString() ,
Command.Program , Command.Name , Command.ParameterList , Command.Parameters ,
Command.Paramsint , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Fields

conditionResult

Stores the result of the evaluated condition.

```
public bool conditionResult
```

Field Value

[bool](#)

lhs

Left-hand side of the loop condition.

```
public string lhs
```

Field Value

[string](#)

op

Comparison operator used in the loop condition. Supported operators are: <, >, <=, >=, ==, !=.

```
public string op
```

Field Value

[string](#)

program

```
public StoredProgram program
```

Field Value

StoredProgram

rhs

Right-hand side of the loop condition.

```
public string rhs
```

Field Value

[string](#) 

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public override void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#) 

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not three parameters stored as expected.

Compile()

Compiles if command by validating the comparison operator.

```
public override void Compile()
```

Exceptions

ParserException

Thrown when an invalid operator is supplied.

EvaluateCondition()

Evaluates the conditional expression by resolving operators and applying the comparison operator.

```
public bool EvaluateCondition()
```

Returns

[bool](#)

true if the condition evaluates to true; otherwise **false**.

Exceptions

CommandException

Thrown when an invalid operator is encountered at runtime.

Execute()

Executes while command by evaluating the loop condition and determining whether to enter or exit the loop body.

```
public override void Execute()
```

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running checkparameters method.

```
public override void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).

Class AppWhile.WhileRuntime

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Maintains a stack of active while commands.

```
public static class AppWhile.WhileRuntime
```

Inheritance

[object](#)  ← AppWhile.WhileRuntime

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Properties

Stack

```
public static Stack<AppWhile> Stack { get; }
```

Property Value

[Stack](#)  <[AppWhile](#)>

Class CircleCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for drawing circles onto canvas at current pen position. Bypasses BOOSE restrictions. Uses BOOSE.ICommand interface.

```
public class CircleCommand : ICommand
```

Inheritance

[object](#) ← CircleCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Remarks

This command stores its radius parameter as a string during parsing so it can be evaluated at runtime. The actual drawing is performed by calling [Circle\(int, bool\)](#).

Constructors

CircleCommand(AppCanvas)

Constructor for command setting up canvas for drawing.

```
public CircleCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

The canvas that will receive the drawing.

Fields

canvas

Canvas that will be drawn onto.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

filled

Determines whether circle will be filled.

```
public bool filled
```

Field Value

[bool](#)

program

Stored commands passed by parser for execution.

```
public StoredProgram program
```

Field Value

StoredProgram

radiusParam

The radius of the circle passed by the parser as a string for evaluation.

```
public string radiusParam
```

Field Value

[string](#)

Properties

radius

The radius of the circle to be drawn set during execution.

```
public int radius { get; set; }
```

Property Value

[int](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown is there is not one parameters stored as expected.

Compile()

Validates parameter during compilation if raw integer is passed (expressions are validated during execution).

```
public void Compile()
```

Remarks

If either parameter is an expression or variable reference, full validation is deferred until [Execute\(\)](#).

Exceptions

ParseException

Thrown when the radius is a literal integer less than zero.

Execute()

Executes circle command by calling canvas method to draw after handling radius argument passed by parser.

```
public void Execute()
```

Remarks

If [radiusParam](#) is a literal integer, it is used directly. Otherwise, it is evaluated using [Evaluate Expression\(string\)](#), rounded to the nearest integer, and then used as the radius.

Set(StoredProgram, string)

Receives raw parser input then sorts it into a command and its parameters. Validates parameter count by running checkParameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)

Reference for command parameters stored by the parser.

Class CommandParser

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Parses user input into valid BOOSE commands. Works with command factory to create command objects and stores them in BOOSE.StoredProgram for execution.

```
public class CommandParser : IParser
```

Inheritance

[object](#) ← CommandParser

Implements

IParser

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

CommandParser(AppCommandFactory, StoredProgram)

Constructor for [CommandParser](#) that initialises command factory and stored program.

```
public CommandParser(AppCommandFactory factory, StoredProgram program)
```

Parameters

factory [AppCommandFactory](#)

Factory used to create commands.

program StoredProgram

Program used to store commands.

Fields

factory

Factory used to create command objects based on parsed user input.

```
public AppCommandFactory factory
```

Field Value

[AppCommandFactory](#).

storedProgram

Stores command objects so they can be executed.

```
public StoredProgram storedProgram
```

Field Value

StoredProgram

Methods

ParseCommand(string)

Parses a single line of user input into a command object.

```
public ICommand ParseCommand(string Line)
```

Parameters

Line [string](#) 

Raw user input string.

Returns

ICommand

A compiled BOOSE.ICommand object containing command and command parameters for execution.

Exceptions

ParserException

Class DrawToCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for drawing a line on canvas from current pen position.

```
public class DrawToCommand : ICommand
```

Inheritance

[object](#)  ← DrawToCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

DrawToCommand(AppCanvas)

Constructor for command that sets up canvas for drawing.

```
public DrawToCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

Fields

canvas

Canvas to be drawn onto.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

program

Command stored by parser.

```
public StoredProgram program
```

Field Value

StoredProgram

X

x and y co-ordinates of pen position on canvas.

```
public int x
```

Field Value

[int](#)

xParam

X and Y coordinate parameters passed by the parser for evaluation.

```
public string xParam
```

Field Value

[string](#)

y

x and y co-ordinates of pen position on canvas.

```
public int y
```

Field Value

[int](#)

yParam

X and Y coordinate parameters passed by the parser for evaluation.

```
public string yParam
```

Field Value

[string](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown is there is not two parameters stored as expected.

Compile()

Validates parameters at compile time when literal integers are provided.

```
public void Compile()
```

Remarks

If either parameter is an expression or variable reference, full validation is deferred until [Execute\(\)](#).

Exceptions

ParserException

Thrown if a literal X or Y value is negative.

Execute()

Executes the command by resolving X and Y values and drawing a line to the target coordinate.

```
public void Execute()
```

Remarks

If a parameter is a literal integer it is used directly; otherwise it is evaluated using [EvaluateExpressionWithString\(string\)](#) and parsed to an integer.

Exceptions

CommandException

Thrown if the resolved X or Y value is negative.

Set(StoredProgram, string)

Receives raw parser input then sorts it into a command and its parameters. Validates parameter count by running checkParameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Class Form1

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

[Form1](#) operates as the main class for the application. It provides a windows form GUI to store and execute user commands, which interact with the drawable canvas.

```
public class Form1 : Form, IDropTarget, ISynchronizeInvoke, IWin32Window,
    IBindableComponent, IComponent, IDisposable, IContainerControl
```

Inheritance

[object](#)  ← [MarshalByRefObject](#)  ← [Component](#)  ← [Control](#)  ← [ScrollableControl](#)  ← [ContainerControl](#)  ← [Form](#)  ← Form1

Implements

[IDropTarget](#) , [ISynchronizeInvoke](#) , [IWin32Window](#) , [IBindableComponent](#) , [IComponent](#) , [IDisposable](#) , [IContainerControl](#) 

Inherited Members

[Form.SetVisibleCore\(bool\)](#) , [Form.Activate\(\)](#) , [Form.ActivateMdiChild\(Form\)](#) , [Form.AddOwnedForm\(Form\)](#) , [Form.AdjustFormScrollbars\(bool\)](#) , [Form.Close\(\)](#) , [Form.CreateAccessibilityInstance\(\)](#) , [Form.CreateControlsInstance\(\)](#) , [Form.CreateHandle\(\)](#) , [Form.DefWndProc\(ref Message\)](#) , [Form.ProcessMnemonic\(char\)](#) , [Form.CenterToParent\(\)](#) , [Form.CenterToScreen\(\)](#) , [Form.LayoutMdi\(MdiLayout\)](#) , [Form.OnActivated\(EventArgs\)](#) , [Form.OnBackgroundImageChanged\(EventArgs\)](#) , [Form.OnBackgroundImageLayoutChanged\(EventArgs\)](#) , [Form.OnClosing\(CancelEventArgs\)](#) , [Form.OnClosed\(EventArgs\)](#) , [Form.OnFormClosing\(FormClosingEventArgs\)](#) , [Form.OnFormClosed\(FormClosedEventArgs\)](#) , [Form.OnCreateControl\(\)](#) , [Form.OnDeactivate\(EventArgs\)](#) , [Form.OnEnabledChanged\(EventArgs\)](#) , [Form.OnEnter\(EventArgs\)](#) , [Form.OnFontChanged\(EventArgs\)](#) , [Form.OnGotFocus\(EventArgs\)](#) , [Form.OnHandleCreated\(EventArgs\)](#) , [Form.OnHandleDestroyed\(EventArgs\)](#) , [Form.OnHelpButtonClicked\(CancelEventArgs\)](#) , [Form.OnLayout\(LayoutEventArgs\)](#) , [Form.OnLoad\(EventArgs\)](#) , [Form.OnMaximizedBoundsChanged\(EventArgs\)](#) , [Form.OnMaximumSizeChanged\(EventArgs\)](#) , [Form.OnMinimumSizeChanged\(EventArgs\)](#) , [Form.OnInputLanguageChanged\(InputLanguageChangedEventArgs\)](#) , [Form.OnInputLanguageChanging\(InputLanguageChangingEventArgs\)](#) , [Form.OnVisibleChanged\(EventArgs\)](#) , [Form.OnMdiChildActivate\(EventArgs\)](#) , [Form.OnMenuStart\(EventArgs\)](#) , [Form.OnMenuComplete\(EventArgs\)](#) , [Form.OnPaint\(PaintEventArgs\)](#) , [Form.OnResize\(EventArgs\)](#) 

[Form.OnDpiChanged\(DpiChangedEventArgs\)](#) , [Form.OnGetDpiScaledSize\(int, int, ref Size\)](#) ,
[Form.OnRightToLeftLayoutChanged\(EventArgs\)](#) , [Form.OnShown\(EventArgs\)](#) ,
[Form.OnTextChanged\(EventArgs\)](#) , [Form.ProcessCmdKey\(ref Message, Keys\)](#) ,
[Form.ProcessDialogKey\(Keys\)](#) , [Form.ProcessDialogChar\(char\)](#) ,
[Form.ProcessKeyPreview\(ref Message\)](#) , [Form.ProcessTabKey\(bool\)](#) ,
[Form.RemoveOwnedForm\(Form\)](#) , [Form.Select\(bool, bool\)](#) ,
[Form.ScaleMinMaxSize\(float, float, bool\)](#) ,
[Form.GetScaledBounds\(Rectangle, SizeF, BoundsSpecified\)](#) ,
[Form.ScaleControl\(SizeF, BoundsSpecified\)](#) , [Form.SetBoundsCore\(int, int, int, int, BoundsSpecified\)](#) ,
[Form.SetClientSizeCore\(int, int\)](#) , [Form.SetDesktopBounds\(int, int, int, int\)](#) ,
[Form.SetDesktopLocation\(int, int\)](#) , [Form.Show\(IWin32Window\)](#) , [Form.ShowDialog\(\)](#) ,
[Form.ShowDialog\(IWin32Window\)](#) , [Form.ToString\(\)](#) , [Form.UpdateDefaultButton\(\)](#) ,
[Form.OnResizeBegin\(EventArgs\)](#) , [Form.OnResizeEnd\(EventArgs\)](#) ,
[Form.OnStyleChanged\(EventArgs\)](#) , [Form.ValidateChildren\(\)](#) ,
[Form.ValidateChildren\(ValidationConstraints\)](#) , [Form.WndProc\(ref Message\)](#) , [Form.AcceptButton](#) ,
[Form.ActiveForm](#) , [Form.ActiveMdiChild](#) , [Form.AllowTransparency](#) , [Form.AutoScroll](#) ,
[Form.AutoSize](#) , [Form.AutoSizeMode](#) , [Form.AutoValidate](#) , [Form.BackColor](#) ,
[Form.FormBorderStyle](#) , [Form.CancelButton](#) , [Form.ClientSize](#) , [Form.ControlBox](#) ,
[Form.CreateParams](#) , [Form.DefaultImeMode](#) , [Form.DefaultSize](#) , [Form.DesktopBounds](#) ,
[Form.DesktopLocation](#) , [Form.DialogResult](#) , [Form.HelpButton](#) , [Form.Icon](#) , [Form.IsMdiChild](#) ,
[Form.IsMdiContainer](#) , [Form.IsRestrictedWindow](#) , [Form.KeyPreview](#) , [Form.Location](#) ,
[Form.MaximizedBounds](#) , [Form.MaximumSize](#) , [Form.MainMenuStrip](#) , [Form.MinimumSize](#) ,
[Form.MaximizeBox](#) , [Form.MdiChildren](#) , [Form.MdiChildrenMinimizedAnchorBottom](#) ,
[Form.MdiParent](#) , [Form.MinimizeBox](#) , [Form.Modal](#) , [Form.Opacity](#) , [Form.OwnedForms](#) ,
[Form.Owner](#) , [Form.RestoreBounds](#) , [Form.RightToLeftLayout](#) , [Form.ShowInTaskbar](#) ,
[Form.ShowIcon](#) , [Form.ShowWithoutActivation](#) , [Form.Size](#) , [Form.SizeGripStyle](#) ,
[Form.StartPosition](#) , [Form.Text](#) , [Form.TopLevel](#) , [Form.TopMost](#) , [Form.TransparencyKey](#) ,
[Form.WindowState](#) , [Form.AutoSizeChanged](#) , [Form.AutoValidateChanged](#) ,
[Form.HelpButtonClicked](#) , [Form.MaximizedBoundsChanged](#) , [Form.MaximumSizeChanged](#) ,
[Form.MinimumSizeChanged](#) , [Form.Activated](#) , [Form.Deactivate](#) , [Form.FormClosing](#) ,
[Form.FormClosed](#) , [Form.Load](#) , [Form.MdiChildActivate](#) , [Form.MenuComplete](#) ,
[Form.MenuStart](#) , [Form.InputLanguageChanged](#) , [Form.InputLanguageChanging](#) ,
[Form.RightToLeftLayoutChanged](#) , [Form.Shown](#) , [Form.DpiChanged](#) , [Form.ResizeBegin](#) ,
[Form.ResizeEnd](#) , [ContainerControl.OnAutoValidateChanged\(EventArgs\)](#) ,
[ContainerControl.OnMove\(EventArgs\)](#) , [ContainerControl.OnParentChanged\(EventArgs\)](#) ,
[ContainerControl.PerformAutoScale\(\)](#) , [ContainerControl.RescaleConstantsForDpi\(int, int\)](#) ,
[ContainerControl.Validate\(\)](#) , [ContainerControl.Validate\(bool\)](#) ,
[ContainerControl.AutoScaleDimensions](#) , [ContainerControl.AutoScaleFactor](#) ,
[ContainerControl.AutoScaleMode](#) , [ContainerControl.BindingContext](#) ,
[ContainerControl.CanEnableIme](#) , [ContainerControl.ActiveControl](#) ,

[ContainerControl.CurrentAutoScaleDimensions](#) , [ContainerControl.ParentForm](#) ,
[ScrollableControl.ScrollStateAutoScrolling](#) , [ScrollableControl.ScrollStateHScrollVisible](#) ,
[ScrollableControl.ScrollStateVScrollVisible](#) , [ScrollableControl.ScrollStateUserHasScrolled](#) ,
[ScrollableControl.ScrollStateFullDrag](#) , [ScrollableControl.GetScrollState\(int\)](#) ,
[ScrollableControl.OnMouseWheel\(MouseEventArgs\)](#) ,
[ScrollableControl.OnRightToLeftChanged\(EventArgs\)](#) ,
[ScrollableControl.OnPaintBackground\(PaintEventArgs\)](#) ,
[ScrollableControl.OnPaddingChanged\(EventArgs\)](#) , [ScrollableControl.SetDisplayRectLocation\(int, int\)](#) ,
[ScrollableControl.ScrollControlIntoView\(Control\)](#) , [ScrollableControl.ScrollToControl\(Control\)](#) ,
[ScrollableControl.OnScroll\(ScrollEventArgs\)](#) , [ScrollableControl.SetAutoScrollMargin\(int, int\)](#) ,
[ScrollableControl.SetScrollState\(int, bool\)](#) , [ScrollableControl.AutoScrollMargin](#) ,
[ScrollableControl.AutoScrollPosition](#) , [ScrollableControl.AutoScrollMinSize](#) ,
[ScrollableControl.DisplayRectangle](#) , [ScrollableControl.HScroll](#) , [ScrollableControl.HorizontalScroll](#) ,
[ScrollableControl.VScroll](#) , [ScrollableControl.VerticalScroll](#) , [ScrollableControl.Scroll](#) ,
[Control.GetAccessibilityObjectById\(int\)](#) , [Control.SetAutoSizeMode\(AutoSizeMode\)](#) ,
[Control.GetAutoSizeMode\(\)](#) , [Control.GetPreferredSize\(Size\)](#) ,
[Control.AccessibilityNotifyClients\(AccessibleEvents, int\)](#) ,
[Control.AccessibilityNotifyClients\(AccessibleEvents, int, int\)](#) , [Control.BeginInvoke\(Delegate\)](#) ,
[Control.BeginInvoke\(Action\)](#) , [Control.BeginInvoke\(Delegate, params object\[\]\)](#) ,
[Control.BringToFront\(\)](#) , [Control.Contains\(Control\)](#) , [Control.CreateGraphics\(\)](#) ,
[Control.CreateControl\(\)](#) , [Control.DestroyHandle\(\)](#) , [Control.DoDragDrop\(object, DragDropEffects\)](#) ,
[Control.DoDragDrop\(object, DragDropEffects, Bitmap, Point, bool\)](#) ,
[Control.DrawToBitmap\(Bitmap, Rectangle\)](#) , [Control.EndInvoke\(IAsyncResult\)](#) , [Control.FindForm\(\)](#) ,
[Control.GetTopLevel\(\)](#) , [Control.RaiseKeyEvent\(object, KeyEventArgs\)](#) ,
[Control.RaiseMouseEvent\(object, MouseEventArgs\)](#) , [Control.Focus\(\)](#) ,
[Control.FromChildHandle\(nint\)](#) , [Control.FromHandle\(nint\)](#) ,
[Control.GetChildAtPoint\(Point, GetChildAtPointSkip\)](#) , [Control.GetChildAtPoint\(Point\)](#) ,
[Control.GetContainerControl\(\)](#) , [Control.GetNextControl\(Control, bool\)](#) ,
[Control.GetStyle\(ControlStyles\)](#) , [Control.Hide\(\)](#) , [Control.InitLayout\(\)](#) , [Control.Invalidate\(Region\)](#) ,
[Control.Invalidate\(Region, bool\)](#) , [Control.Invalidate\(\)](#) , [Control.Invalidate\(bool\)](#) ,
[Control.Invalidate\(Rectangle\)](#) , [Control.Invalidate\(Rectangle, bool\)](#) , [Control.Invoke\(Action\)](#) ,
[Control.Invoke\(Delegate\)](#) , [Control.Invoke\(Delegate, params object\[\]\)](#) ,
[Control.Invoke<T>\(Func<T>\)](#) , [Control.InvokePaint\(Control, PaintEventArgs\)](#) ,
[Control.InvokePaintBackground\(Control, PaintEventArgs\)](#) , [Control.IsKeyLocked\(Keys\)](#) ,
[Control.IsInputChar\(char\)](#) , [Control.IsInputKey\(Keys\)](#) , [Control.IsMnemonic\(char, string\)](#) ,
[Control.LogicalToDeviceUnits\(int\)](#) , [Control.LogicalToDeviceUnits\(Size\)](#) ,
[Control.ScaleBitmapLogicalToDevice\(ref Bitmap\)](#) , [Control.NotifyInvalidate\(Rectangle\)](#) ,
[Control.InvokeOnClick\(Control, EventArgs\)](#) , [Control.OnAutoSizeChanged\(EventArgs\)](#) ,
[Control.OnBackColorChanged\(EventArgs\)](#) , [Control.OnBindingContextChanged\(EventArgs\)](#) ,
[Control.OnCausesValidationChanged\(EventArgs\)](#) , [Control.OnContextMenuStripChanged\(EventArgs\)](#) ,

[Control.OnCursorChanged\(EventArgs\)](#), [Control.OnDataContextChanged\(EventArgs\)](#),
[Control.OnDockChanged\(EventArgs\)](#), [Control.OnForeColorChanged\(EventArgs\)](#),
[Control.OnNotifyMessage\(Message\)](#), [Control.OnParentBackColorChanged\(EventArgs\)](#),
[Control.OnParentBackgroundImageChanged\(EventArgs\)](#),
[Control.OnParentBindingContextChanged\(EventArgs\)](#), [Control.OnParentCursorChanged\(EventArgs\)](#),
[Control.OnParentDataContextChanged\(EventArgs\)](#), [Control.OnParentEnabledChanged\(EventArgs\)](#),
[Control.OnParentFontChanged\(EventArgs\)](#), [Control.OnParentForeColorChanged\(EventArgs\)](#),
[Control.OnParentRightToLeftChanged\(EventArgs\)](#), [Control.OnParentVisibleChanged\(EventArgs\)](#),
[Control.OnPrint\(PaintEventArgs\)](#), [Control.OnTabIndexChanged\(EventArgs\)](#),
[Control.OnTabStopChanged\(EventArgs\)](#), [Control.OnClick\(EventArgs\)](#),
[Control.OnClientSizeChanged\(EventArgs\)](#), [Control.OnControlAdded\(ControlEventArgs\)](#),
[Control.OnControlRemoved\(ControlEventArgs\)](#), [Control.OnLocationChanged\(EventArgs\)](#),
[Control.OnDoubleClick\(EventArgs\)](#), [Control.OnDragEnter\(DragEventArgs\)](#),
[Control.OnDragOver\(DragEventArgs\)](#), [Control.OnDragLeave\(EventArgs\)](#),
[Control.OnDragDrop\(DragEventArgs\)](#), [Control.OnGiveFeedback\(GiveFeedbackEventArgs\)](#),
[Control.InvokeGotFocus\(Control, EventArgs\)](#), [Control.OnHelpRequested\(HelpEventArgs\)](#),
[Control.OnInvalidated\(InvalidateEventArgs\)](#), [Control.OnKeyDown\(KeyEventArgs\)](#),
[Control.OnKeyPress\(KeyPressEventArgs\)](#), [Control.OnKeyUp\(KeyEventArgs\)](#),
[Control.OnLeave\(EventArgs\)](#), [Control.InvokeLostFocus\(Control, EventArgs\)](#),
[Control.OnLostFocus\(EventArgs\)](#), [Control.OnMarginChanged\(EventArgs\)](#),
[Control.OnMouseDoubleClick\(MouseEventArgs\)](#), [Control.OnMouseClick\(MouseEventArgs\)](#),
[Control.OnMouseCaptureChanged\(EventArgs\)](#), [Control.OnMouseDown\(MouseEventArgs\)](#),
[Control.OnMouseEnter\(EventArgs\)](#), [Control.OnMouseLeave\(EventArgs\)](#),
[Control.OnDpiChangedBeforeParent\(EventArgs\)](#), [Control.OnDpiChangedAfterParent\(EventArgs\)](#),
[Control.OnMouseHover\(EventArgs\)](#), [Control.OnMouseMove\(MouseEventArgs\)](#),
[Control.OnMouseUp\(MouseEventArgs\)](#),
[Control.OnQueryContinueDrag\(QueryContinueDragEventArgs\)](#),
[Control.OnRegionChanged\(EventArgs\)](#), [Control.OnPreviewKeyDown\(PreviewKeyDownEventArgs\)](#),
[Control.OnSizeChanged\(EventArgs\)](#), [Control.OnChangeUICues\(UICuesEventArgs\)](#),
[Control.OnSystemColorsChanged\(EventArgs\)](#), [Control.OnValidating\(CancelEventArgs\)](#),
[Control.OnValidated\(EventArgs\)](#), [Control.PerformLayout\(\)](#), [Control.PerformLayout\(Control, string\)](#),
[Control.PointToClient\(Point\)](#), [Control.PointToScreen\(Point\)](#),
[Control.PreProcessMessage\(ref Message\)](#), [Control.PreProcessControlMessage\(ref Message\)](#),
[Control.ProcessKeyEventArgs\(ref Message\)](#), [Control.ProcessKeyMessage\(ref Message\)](#),
[Control.RaiseDragEvent\(object, DragEventArgs\)](#), [Control.RaisePaintEvent\(object, PaintEventArgs\)](#),
[Control.RecreateHandle\(\)](#), [Control.RectangleToClient\(Rectangle\)](#),
[Control.RectangleToScreen\(Rectangle\)](#), [Control.ReflectMessage\(nint, ref Message\)](#),
[Control.Refresh\(\)](#), [Control.ResetMouseEventArgs\(\)](#), [Control.ResetText\(\)](#), [Control.ResumeLayout\(\)](#),
[Control.ResumeLayout\(bool\)](#), [Control.Scale\(SizeF\)](#), [Control.Select\(\)](#),
[Control.SelectNextControl\(Control, bool, bool, bool, bool\)](#), [Control.SendToBack\(\)](#),

[Control.SetBounds\(int, int, int, int\)](#), [Control.SetBounds\(int, int, int, int, BoundsSpecified\)](#), [Control.SizeFromClientSize\(Size\)](#), [Control.SetStyle\(ControlStyles, bool\)](#), [Control.SetTopLevel\(bool\)](#), [Control.RtlTranslateAlignment\(HorizontalAlignment\)](#), [Control.RtlTranslateAlignment\(LeftRightAlignment\)](#), [Control.RtlTranslateAlignment\(ContentAlignment\)](#), [Control.RtlTranslateHorizontal\(HorizontalAlignment\)](#), [Control.RtlTranslateLeftRight\(LeftRightAlignment\)](#), [Control.RtlTranslateContent\(ContentAlignment\)](#), [Control.Show\(\)](#), [Control.SuspendLayout\(\)](#), [Control.Update\(\)](#), [Control.UpdateBounds\(\)](#), [Control.UpdateBounds\(int, int, int, int\)](#), [Control.UpdateBounds\(int, int, int, int, int, int\)](#), [Control.UpdateZOrder\(\)](#), [Control.UpdateStyles\(\)](#), [Control.OnImeModeChanged\(EventArgs\)](#), [Control.AccessibilityObject](#), [Control.AccessibleDefaultActionDescription](#), [Control.AccessibleDescription](#), [Control.AccessibleName](#), [Control.AccessibleRole](#), [Control.AllowDrop](#), [Control.Anchor](#), [Control.AutoScrollOffset](#), [Control.LayoutEngine](#), [Control.DataContext](#), [Control.BackgroundImage](#), [Control.BackgroundImageLayout](#), [Control.Bottom](#), [Control.Bounds](#), [Control.CanFocus](#), [Control.CanRaiseEvents](#), [Control.CanSelect](#), [Control.Capture](#), [Control.CausesValidation](#), [Control.CheckForIllegalCrossThreadCalls](#), [Control.ClientRectangle](#), [Control.CompanyName](#), [Control.ContainsFocus](#), [Control.ContextMenuStrip](#), [Control.Controls](#), [Control.Created](#), [Control.Cursor](#), [Control.DataBindings](#), [Control.DefaultBackColor](#), [Control.DefaultCursor](#), [Control.DefaultFont](#), [Control.DefaultForeColor](#), [Control.DefaultMargin](#), [Control.DefaultMaximumSize](#), [Control.DefaultMinimumSize](#), [Control.DefaultPadding](#), [Control.DeviceDpi](#), [Control.IsDisposed](#), [Control.Disposing](#), [Control.Dock](#), [Control.DoubleBuffered](#), [Control.Enabled](#), [Control.Focused](#), [Control.Font](#), [Control.FontHeight](#), [Control.ForeColor](#), [Control.Handle](#), [Control.HasChildren](#), [Control.Height](#), [Control.IsHandleCreated](#), [Control.InvokeRequired](#), [Control.IsAccessible](#), [Control.IsAncestorSiteInDesignMode](#), [Control.IsMirrored](#), [Control.Left](#), [Control.Margin](#), [Control.ModifierKeys](#), [Control.MouseButtons](#), [Control.MousePosition](#), [Control.Name](#), [Control.Parent](#), [Control.ProductName](#), [Control.ProductVersion](#), [Control.RecreatingHandle](#), [Control.Region](#), [Control.RenderRightToLeft](#), [Control.ResizeRedraw](#), [Control.Right](#), [Control.RightToLeft](#), [Control.ScaleChildren](#), [Control.Site](#), [Control.TabIndex](#), [Control.TabStop](#), [Control.Tag](#), [Control.Top](#), [Control.TopLevelControl](#), [Control.ShowKeyboardCues](#), [Control.ShowFocusCues](#), [Control.UseWaitCursor](#), [Control.Visible](#), [Control.Width](#), [Control.PreferredSize](#), [Control.Padding](#), [Control.ImeMode](#), [Control.ImeModeBase](#), [Control.PropagatingImeMode](#), [Control.BackColorChanged](#), [Control.BackgroundImageChanged](#), [Control.BackgroundImageLayoutChanged](#), [Control.BindingContextChanged](#), [Control.CausesValidationChanged](#), [Control.ClientSizeChanged](#), [Control.ContextMenuStripChanged](#), [Control.CursorChanged](#), [Control.DockChanged](#), [Control.EnabledChanged](#), [Control.FontChanged](#), [Control.ForeColorChanged](#), [Control.LocationChanged](#), [Control.MarginChanged](#), [Control.RegionChanged](#), [Control.RightToLeftChanged](#), [Control.SizeChanged](#), [Control.TabIndexChanged](#),

[Control.TabStopChanged](#) , [Control.TextChanged](#) , [Control.VisibleChanged](#) , [Control.Click](#) , [Control.ControlAdded](#) , [Control.ControlRemoved](#) , [Control.DataContextChanged](#) , [Control.DragDrop](#) , [Control.DragEnter](#) , [Control.DragOver](#) , [Control.DragLeave](#) , [Control.GiveFeedback](#) , [Control.HandleCreated](#) , [Control.HandleDestroyed](#) , [Control.HelpRequested](#) , [Control.Invalidated](#) , [Control.PaddingChanged](#) , [Control.Paint](#) , [Control.QueryContinueDrag](#) , [Control.QueryAccessibilityHelp](#) , [Control.DoubleClick](#) , [Control.Enter](#) , [Control.GotFocus](#) , [Control.KeyDown](#) , [Control.KeyPress](#) , [Control.KeyUp](#) , [Control.Layout](#) , [Control.Leave](#) , [Control.LostFocus](#) , [Control.MouseClick](#) , [Control.MouseDoubleClick](#) , [Control.MouseCaptureChanged](#) , [Control.MouseDown](#) , [Control.MouseEnter](#) , [Control.MouseLeave](#) , [Control.DpiChangedBeforeParent](#) , [Control.DpiChangedAfterParent](#) , [Control.MouseHover](#) , [Control.MouseMove](#) , [Control.MouseUp](#) , [Control.MouseWheel](#) , [Control.Move](#) , [Control.PreviewKeyDown](#) , [Control.Resize](#) , [Control.ChangeUICues](#) , [Control.StyleChanged](#) , [Control.SystemColorsChanged](#) , [Control.Validating](#) , [Control.Validated](#) , [Control.ParentChanged](#) , [Control.ImeModeChanged](#) , [Component.Dispose\(\)](#) , [Component.GetService\(Type\)](#) , [Component.Container](#) , [Component.DesignMode](#) , [Component.Events](#) , [Component.Disposed](#) , [MarshalByRefObject.GetLifetimeService\(\)](#) , [MarshalByRefObject.InitializeLifetimeService\(\)](#) , [MarshalByRefObject.MemberwiseClone\(bool\)](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#)

Constructors

Form1()

Initialises windows GUI form and GUI components. Sets up required BOOSE components and confirms BOOSE is working.

```
public Form1()
```

Methods

Dispose(bool)

Clean up any resources being used.

```
protected override void Dispose(bool disposing)
```

Parameters

disposing [bool](#)

true if managed resources should be disposed; otherwise, false.

Class MoveToCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for moving pen on canvas without drawing.

```
public class MoveToCommand : ICommand
```

Inheritance

[object](#)  ← MoveToCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

MoveToCommand(AppCanvas)

Constructor for moveto command sets up canvas.

```
public MoveToCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

Fields

canvas

Canvas to be operated on.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

program

Command stored by parser.

```
public StoredProgram program
```

Field Value

StoredProgram

X

x and y co-ordinates of pen position on canvas.

```
public int x
```

Field Value

[int](#)

xParam

X and Y coordinate parameters passed by the parser for evaluation.

```
public string xParam
```

Field Value

[string](#)

y

x and y co-ordinates of pen position on canvas.

```
public int y
```

Field Value

[int](#)

yParam

X and Y coordinate parameters passed by the parser for evaluation.

```
public string yParam
```

Field Value

[string](#)

Methods

CheckParameters(string[])

Checks command parameters are valid.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

An array containing command parameters

Exceptions

ParserException

Thrown if there is not two parameters stored as required. Also thrown if the parameters stored are not integers.

Compile()

Validates parameters at compile time when literal integers are provided.

```
public void Compile()
```

Remarks

If either parameter is an expression or variable reference, full validation is deferred until [Execute\(\)](#).

Exceptions

ParserException

Thrown if a literal X or Y value is negative.

Execute()

Executes the command by resolving X and Y values and moving the pen to the target coordinate.

```
public void Execute()
```

Remarks

If a parameter is a literal integer it is used directly; otherwise it is evaluated using [EvaluateExpressionWithString\(string\)](#) and parsed to an integer.

Exceptions

CommandException

Thrown if the resolved X or Y value is negative.

Set(StoredProgram, string)

Receives raw parser input then sorts it into a command and its parameters. Validates parameter count by running checkParameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Class PenCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class used for setting the pen colour used for drawing on canvas.

```
public class PenCommand : ICommand
```

Inheritance

[object](#)  ← PenCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

PenCommand(AppCanvas)

Constructor method including canvas object.

```
public PenCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

Fields

blue

```
public int blue
```

Field Value

[int](#)

blueParam

```
public string blueParam
```

Field Value

[string](#)

canvas

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

green

```
public int green
```

Field Value

[int](#)

greenParam

```
public string greenParam
```

Field Value

[string](#)

program

```
public StoredProgram program
```

Field Value

StoredProgram

red

```
public int red
```

Field Value

[int](#)

redParam

```
public string redParam
```

Field Value

[string](#)

Methods

CheckParameters(string[])

Checks command has correct number of parameters and stores them as strings for parsing.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

An array of command parameters.

Exceptions

ParserException

Thrown if there is not three parameters stored as required. Also thrown if parameters are not integers.

Compile()

```
public void Compile()
```

Execute()

Executes pen command via canvas method.

```
public void Execute()
```

Remarks

Evaluates parameters stored to ensure expressions are correctly handled at run time alongside standard values.

Set(StoredProgram, string)

Sets command parameters.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#) 

Reference for command parameters stored by the parser.

Class RectangleCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for drawing rectangles onto canvas at current pen position. Bypasses BOOSE restrictions.

```
public class RectangleCommand : ICommand
```

Inheritance

[object](#) ← RectangleCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Remarks

Width and height parameters are captured as strings during parsing so they can be evaluated at runtime. The actual drawing is performed by [Rect\(int, int, bool\)](#).

Constructors

RectangleCommand(AppCanvas)

Constructor for command that sets up canvas for drawing.

```
public RectangleCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

The canvas to be drawn onto.

Fields

canvas

Canvas that will be drawn onto.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

filled

Determines whether rectangle is filled or not.

```
public bool filled
```

Field Value

[bool](#)

height

Height and width parameters of rectangle to be drawn set during execution.

```
public int height
```

Field Value

[int](#)

heightParam

Height and width parameters passed by the parser for evaluation during execute method.

```
public string heightParam
```

Field Value

[string](#)

program

Stored commands from parser for execution.

```
public StoredProgram program
```

Field Value

StoredProgram

width

Height and width parameters of rectangle to be drawn set during execution.

```
public int width
```

Field Value

[int](#)

widthParam

Height and width parameters passed by the parser for evaluation during execute method.

```
public string widthParam
```

Field Value

[string](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown is there is not two parameters stored as expected.

Compile()

Validates parameter during compilation if raw integer is passed (expressions are validated during execution).

```
public void Compile()
```

Remarks

If either parameter is an expression or variable reference, full validation is deferred until [Execute\(\)](#).

Exceptions

ParserException

Thrown if a literal width or height value is less than zero.

Execute()

Executes rectangle command by calling canvas method to draw after handling parameter arguments passed by parser.

```
public void Execute()
```

Remarks

If the parameters are literal integers they are used directly. Otherwise they are evaluated using [Evaluate Expression\(string\)](#).[↗]

Exceptions

CommandException

Thrown if value is less than one being invalid for drawing.

Set(StoredProgram, string)

Receives raw parser input then sorts it into a command and its parameters. Validates parameter count by running checkParameters method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program StoredProgram

Reference for the command stored by the parser.

Params [string](#)[↗]

Reference for command parameters stored by the parser.

Class WriteCommand

Namespace: [BOOSE App](#)

Assembly: BOOSE_App.dll

Command class for drawing text onto canvas at current pen position. Uses BOOSE.ICommand interface.

```
public class WriteCommand : ICommand
```

Inheritance

[object](#)  ← WriteCommand

Implements

ICommand

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

WriteCommand(AppCanvas)

Constructor for command that sets up canvas for drawing.

```
public WriteCommand(AppCanvas canvas)
```

Parameters

canvas [AppCanvas](#)

The canvas to be drawn onto.

Fields

canvas

Canvas that will be drawn onto.

```
public AppCanvas canvas
```

Field Value

[AppCanvas](#)

program

Stored commands passed by parser for execution.

```
public StoredProgram program
```

Field Value

StoredProgram

text

The text to be written onto the canvas set during execution.

```
public string text
```

Field Value

[string](#)

textParam

The text to be written onto the canvas passed by the parser for evaluation.

```
public string textParam
```

Field Value

[string](#)

Methods

CheckParameters(string[])

Checks command has correct amount of parameters and stores them as strings for evaluation during execution.

```
public void CheckParameters(string[] Parameters)
```

Parameters

Parameters [string](#)[]

Array containing command parameters.

Exceptions

ParserException

Thrown if there is not one parameter stored as expected.

Compile()

Not implemented.

```
public void Compile()
```

Execute()

Executes write command by calling canvas method after evaluating string passed by parser.

```
public void Execute()
```

Remarks

The command first attempts to evaluate [textParam](#) using [EvaluateExpressionWithString\(string\)](#)[↗]. If evaluation fails, the parameter is treated as an unquoted literal string and written.

Set(StoredProgram, string)

Receives raw parser input then sorts it and validates via setting command parameters and running `checkparameters` method.

```
public void Set(StoredProgram Program, string Params)
```

Parameters

Program `StoredProgram`

Reference for the command stored by the parser.

Params [string](#)[↗]

Reference for command parameters stored by the parser.

Remarks

Parameters are split on commas and whitespace, then validated by [CheckParameters\(string\[\]\)](#).